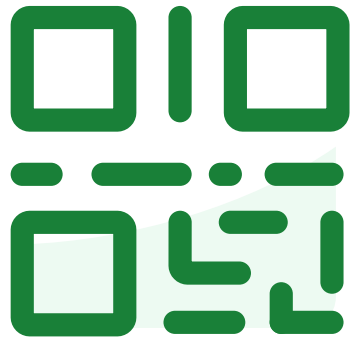


Do not edit
How to change the
design



**Join at slido.com
#1030434**

① The Slido app must be installed on every computer you're presenting from

slido

Lecture 17

Neural Network Regularization & Convolutional Neural Network

Double Descent, Model Averaging, Drop-out, and Residual Connections.
Introduction to convolutional architecture for image analysis

EECS 189/289, Fall 2025 @ UC Berkeley

Joseph E. Gonzalez and Narges Norouzi

Roadmap

- Inductive Biases
- Double Descent on the Bias-Variance Tradeoff
- Model Averaging
- Dropout
- Residual Connections
- Convolutional Neural Networks



1030434

Inductive Biases

- Inductive Biases
- Double Descent on the Bias-Variance Tradeoff
- Model Averaging
- Dropout
- Residual Connections
- Convolutional Neural Networks



1030434



Learning Requires Inductive Biases

“No Free Lunch Theorem” – any learning algorithm is as good as any other when **considering all possible problems**.

- Need to exploit **inductive biases**.

We **encode inductive biases** in neural networks through:

- **Pre-processing:** Transform the data to enforce desired invariances (e.g., normalize scale, orientation, or intensity).
- **Regularized objectives:** Penalize models that exhibit undesired properties during training.
- **Data augmentation:** Expand the training data with transformed examples to make invariances explicit (e.g., rotated images).
- **Architecture design:** Choose model structures that build in invariances (e.g., convolution layers for translation invariance).

Pre-processing through Feature Engineering



Feature Engineering – the process of **manually constructing features** common in many classical machine learning techniques.

Examples:

- **Term Frequency – Inverse Document Frequency (TF-IDF)** augment document representations based on the commonality of words.
- **Histogram of Oriented Gradients (HOG)** was used in computer vision to represent images features.
- **Mel-frequency cepstral coefficients (MFCCs)** are (still) used to represent sound in speech recognition systems.

Modern **deep learning** uses large neural networks as “feature” functions to learn features that are “invariant” to the prediction task.

- Inductive bias shifts to data selection, model, and training process.



Weight Decay Regularization

Previously discussed regularization as an additional penalty $\mathbf{E}_w[\mathbf{w}]$:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \tilde{\mathbf{E}}[\mathbf{w}] = \arg \min_{\mathbf{w}} \mathbf{E}_D[\mathbf{w}, \mathcal{D}] + \lambda \mathbf{E}_w[\mathbf{w}]$$

where λ is the regularization hyper-parameter.

We introduced Ridge-Regression with the L^2 regularization penalty:

$$\mathbf{E}_w[\mathbf{w}] = \frac{1}{2} \sum_{m=1}^M \mathbf{w}_m^2 = \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

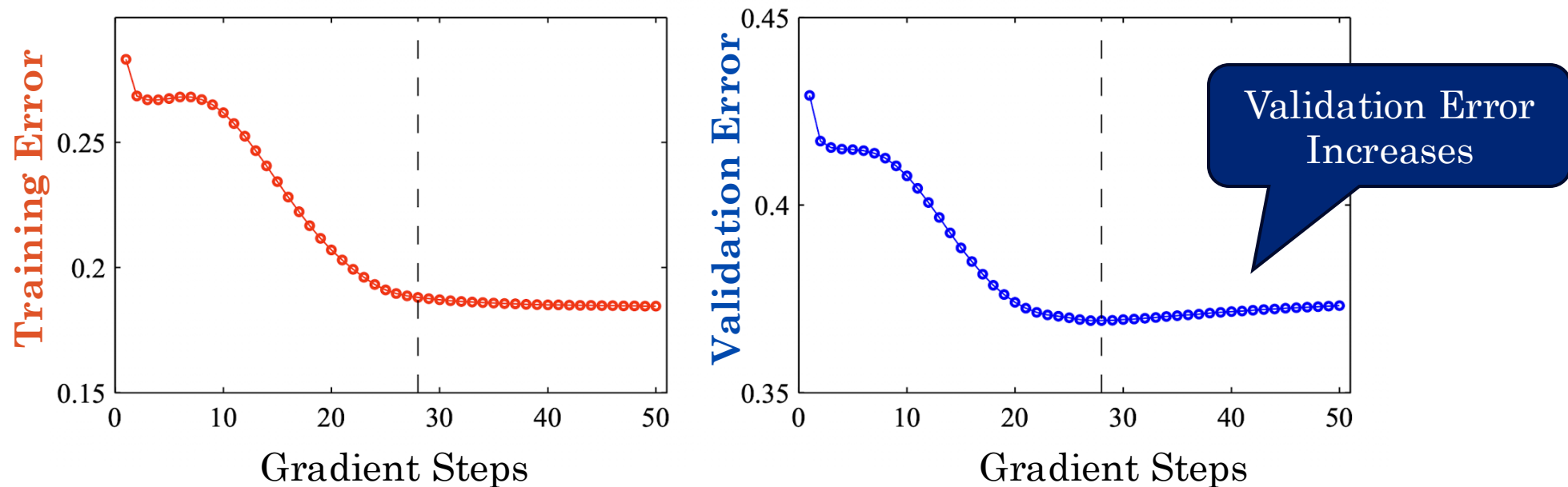
In neural networks, this is called **weight decay** and is commonly applied in the gradient update:

$$\nabla \tilde{\mathbf{E}}[\mathbf{w}] = \nabla \mathbf{E}_D[\mathbf{w}] + \lambda \mathbf{w}$$

Learning Curves and Early Stopping



During training it is common practice to plot a **learning curve** which tracks the **training** and **validation error**



Early Stopping: stop training (return to the model checkpoint) when the validation error stopped decreasing.

Early Stopping is a form of Weight Decay



Because models typically start with relatively small weight values, **stopping gradient descent early** can be viewed as a form of **weight decay**.

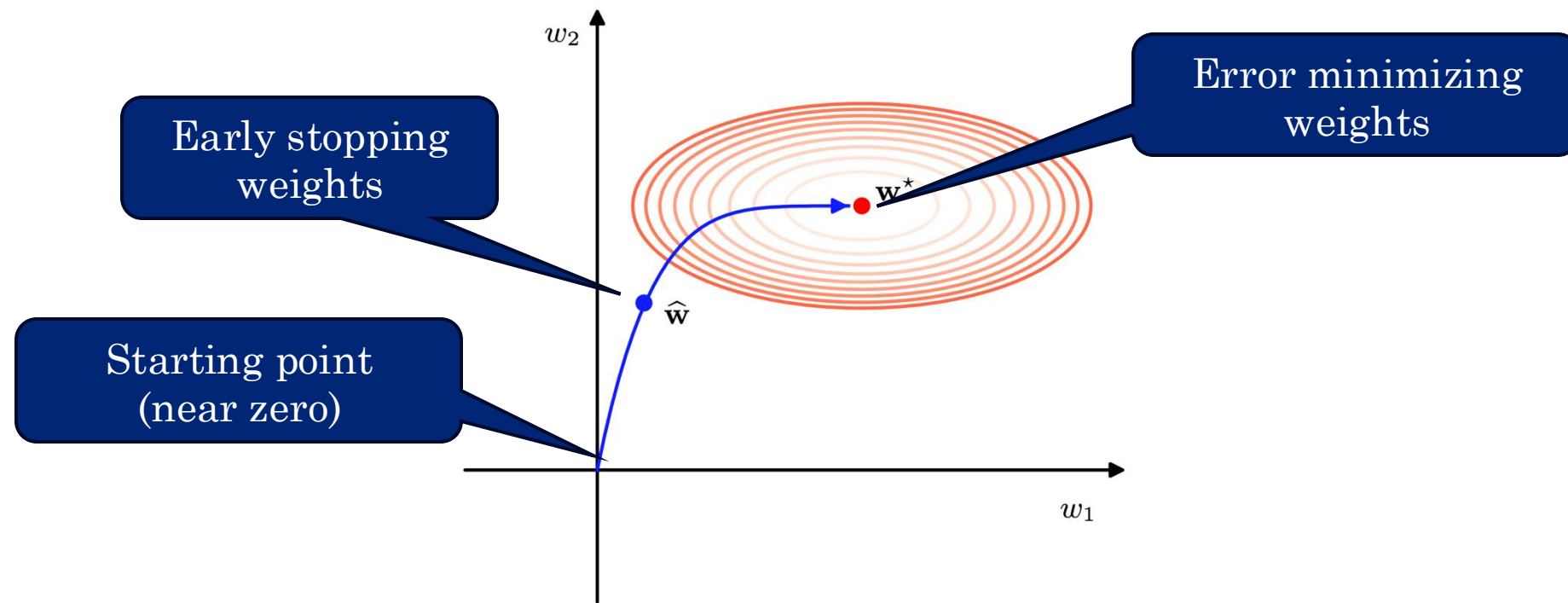


Figure from Bishop Textbook (p268).

Double Descent on the Bias-Variance Tradeoff

- Inductive Biases
- Double Descent on the Bias-Variance Tradeoff
- Model Averaging
- Dropout
- Residual Connections
- Convolutional Neural Networks

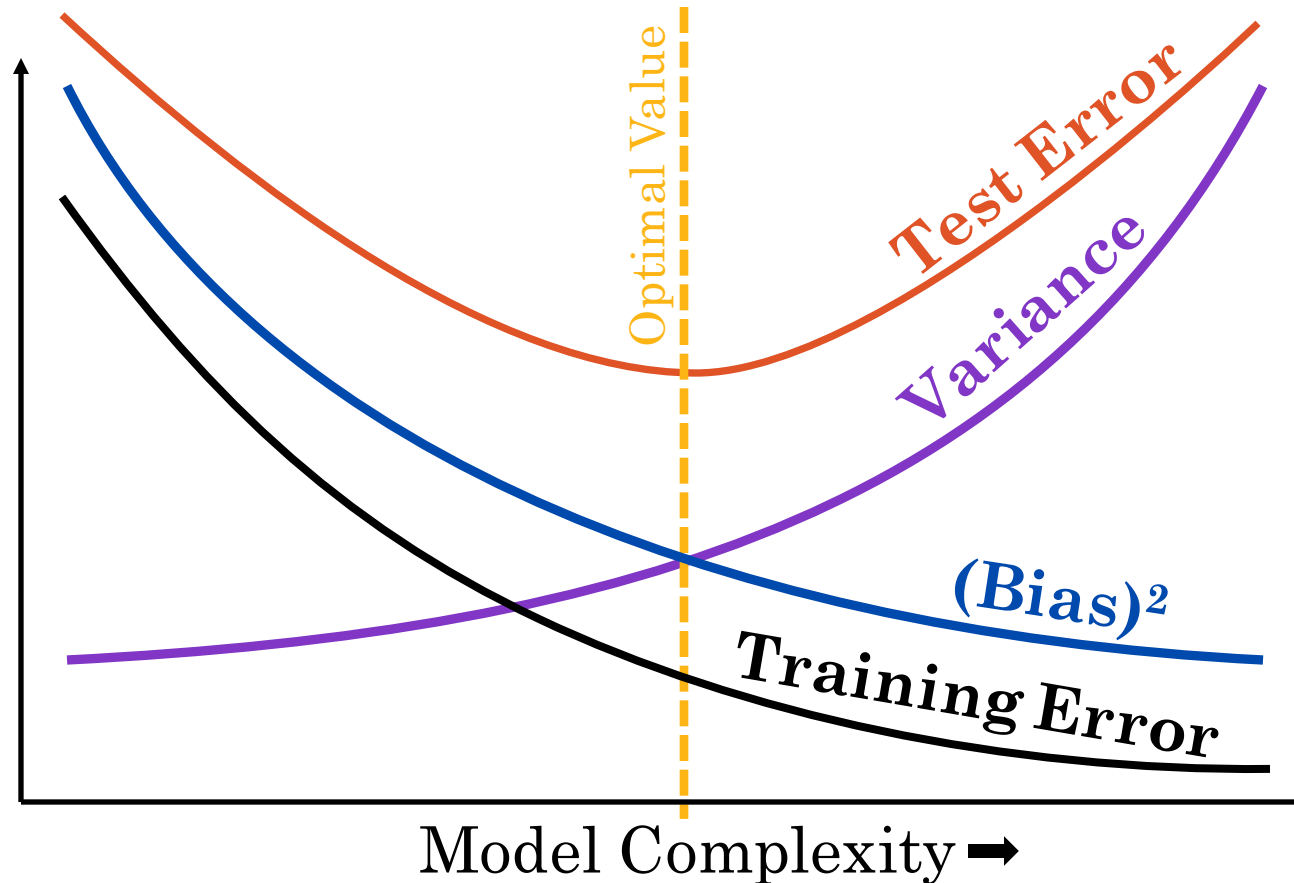


1030434



The Bias Variance Tradeoff Revisited

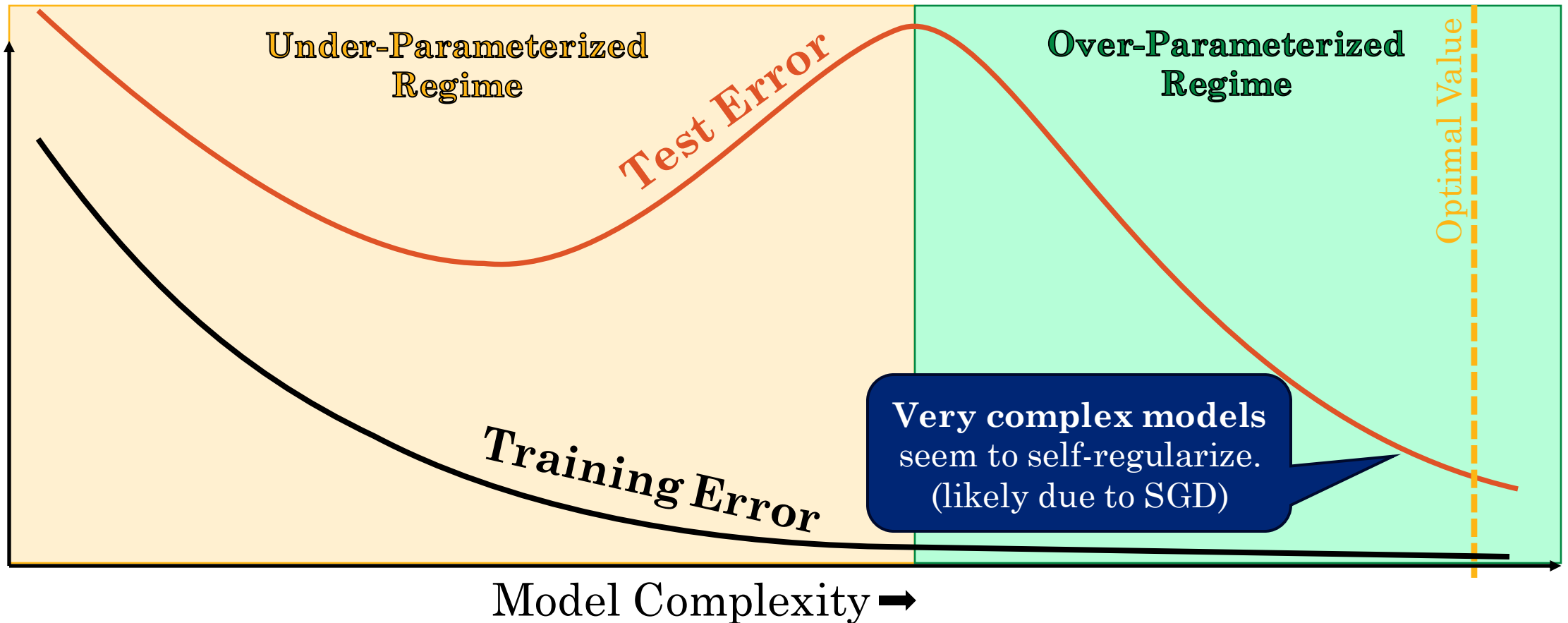
In previous lectures, we introduced the fundamental **bias-variance tradeoff** and how **bias** and **variance** contribute to **test-error**



Double Descent

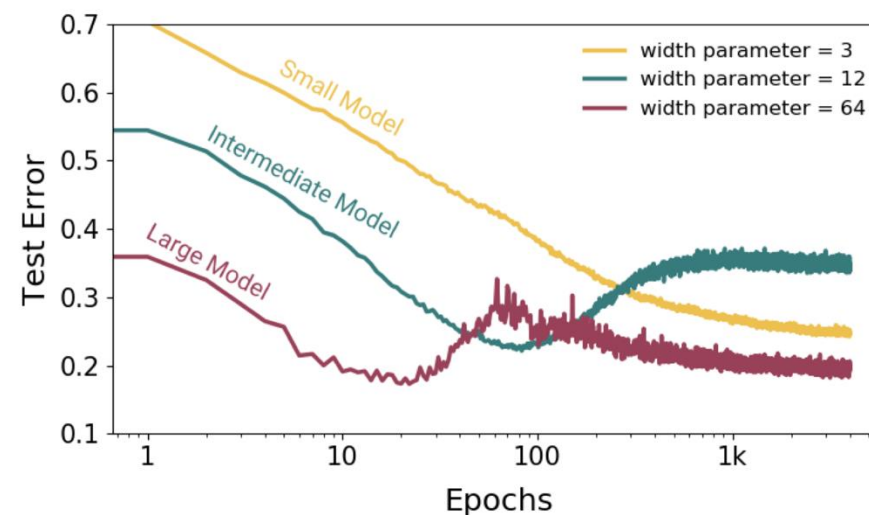
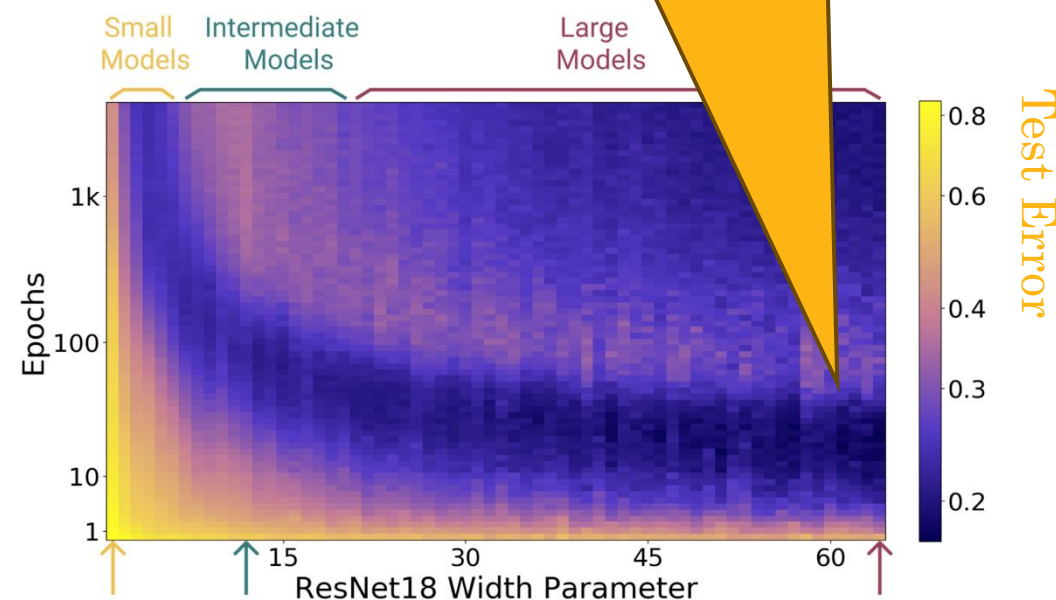
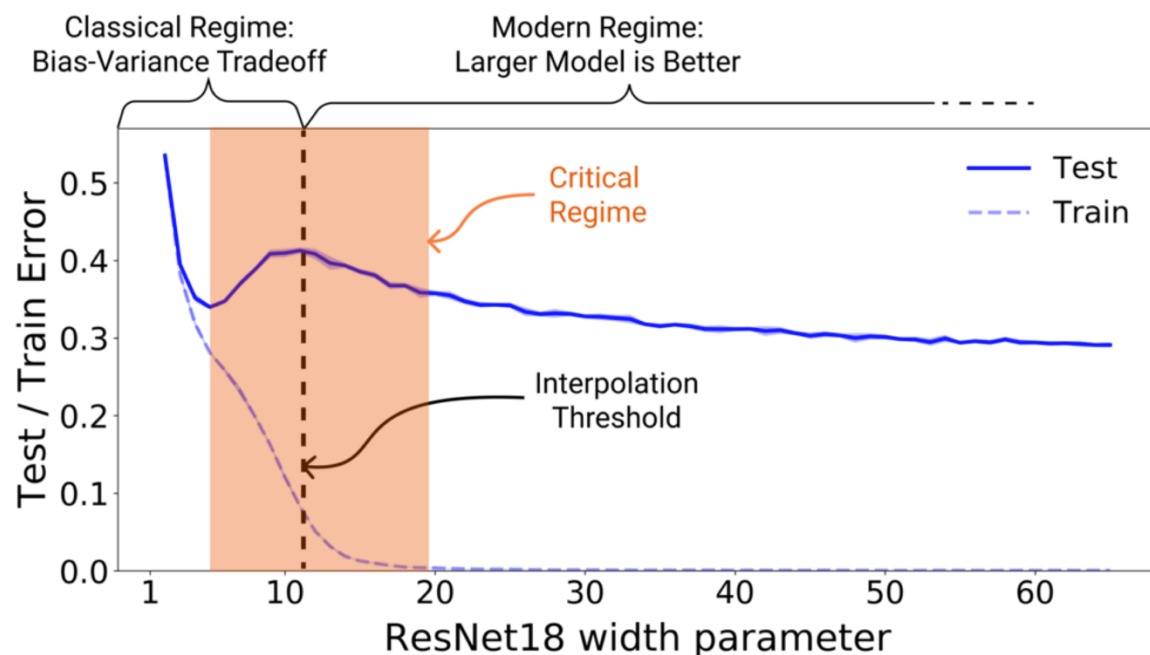


In the case of **deep neural networks trained using SGD**, there is a **second decrease** in test error.



Double Descent in Practice

Examples of double descent
from [Nakkiran et al. ICLR'20](#):



1030434

Model Averaging

- Inductive Biases
- Double Descent on the Bias-Variance Tradeoff
- Model Averaging
- Dropout
- Residual Connections
- Convolutional Neural Networks





Ensembles of Models (Experts)

If you have K models $p_k(y | x)$ for a prediction task $p(y | x)$ then you can often construct an **ensemble model** that **combines the predictions** of each model to **improve accuracy**

$$p_{\text{ens}}(y | x) = \frac{1}{K} \sum_{k=1}^K p_k(y | x)$$

This works by reducing the **prediction variance**:

$$\text{Var}[p_{\text{ens}}(y | x)] = \frac{1}{K^2} \sum_{k=1}^K \text{Var}[p_k(y | x)] = \frac{1}{K^2} \sum_{k=1}^K \sigma^2 = \frac{\sigma^2 K}{K^2} = \frac{\sigma^2}{K}$$

assuming **IID predictions** with **variance** σ^2 .

- Seldom true in practice...



Mixture of Experts (MoEs)

A **mixture of experts** is a **generalization** of the **ensemble method** in which we assign learned weights $\pi(k | x)$ to each expert

$$p_{\text{MoE}}(y | x) = \sum_{k=1}^K \pi(k | x) p_k(y | x)$$

The **mixture weights** $\pi(k | x)$ form a probability distribution and are often sparse (i.e., $\pi(k | x) = 0$ for many k).

- Sometimes called a **routing function** or **routing network**

Mixture of experts (MoEs) are often used as **internal layers** of neural networks as well. (More on this with LLMs)



Constructing Multiple Experts

Ideally, each expert is a **strong independent model**.

- **Strong:** State-of-the-art (**SOTA** ← You should know this word!)
- **Independent:** Trained with different data and inductive biases

Ensembles are constructed by taking **SOTA** models from **competing teams** and **combining them**.

- Ensemble methods topped most classic benchmarks.

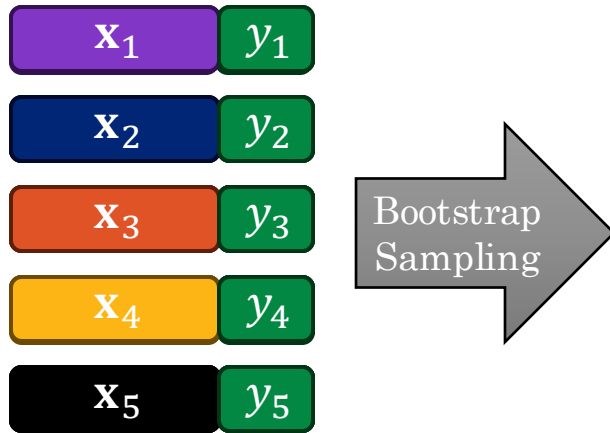
Bootstrap Aggregation (Bagging) is a classic technique to construct an ensemble by training on **bootstrap samples** of the training data.



Bagging: Bootstrap Aggregation

Resample the training data K -times with replacement (**Bootstrap Sampling**) and train a separate model on each sample.

Original Dataset





Bagging: Bootstrap Aggregation

Resample the training data K -times with replacement (**Bootstrap Sampling**) and train a separate model on each sample.

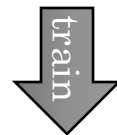
Original Dataset

x_1	y_1
x_2	y_2
x_3	y_3
x_4	y_4
x_5	y_5



Dataset 1

x_1	y_1
x_4	y_4
x_3	y_3
x_4	y_4
x_3	y_3

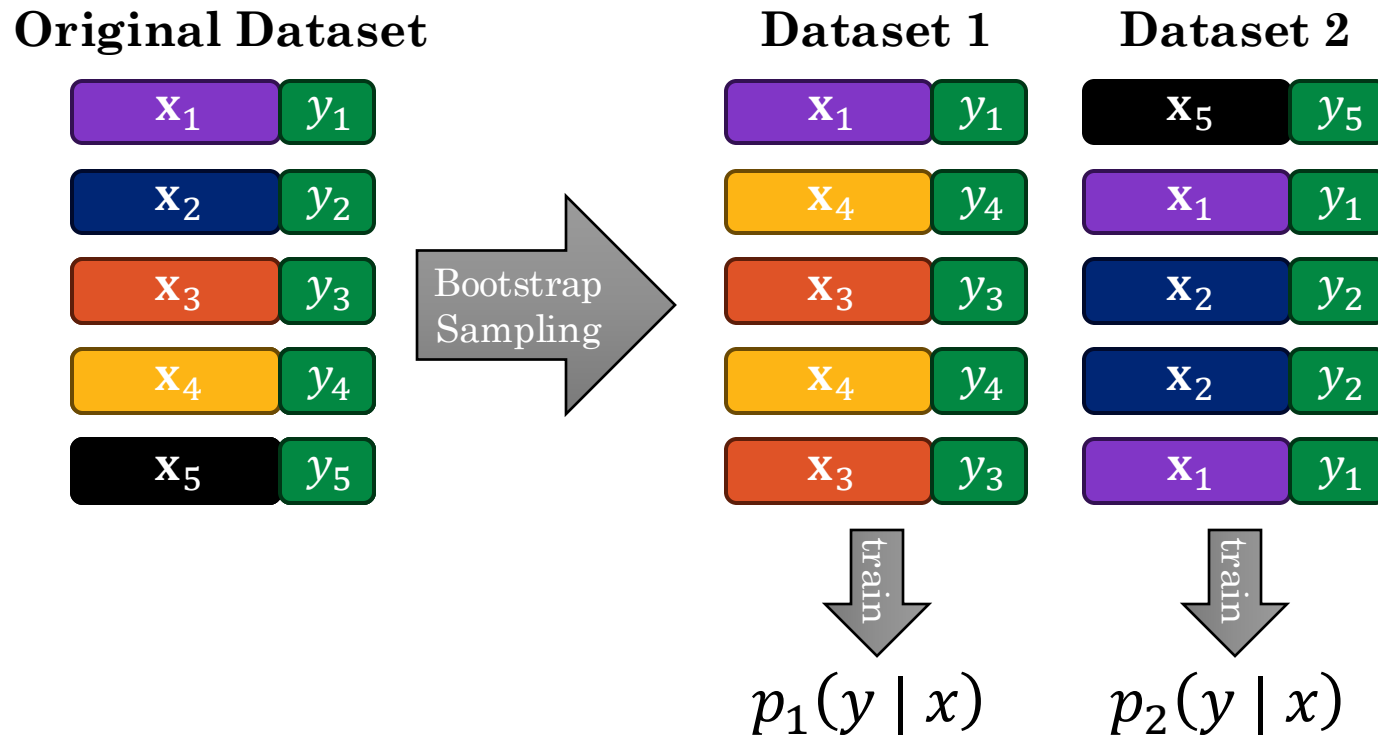


$$p_1(y | x)$$



Bagging: Bootstrap Aggregation

Resample the training data K -times with replacement (**Bootstrap Sampling**) and train a separate model on each sample.





Bagging: Bootstrap Aggregation

Resample the training data K -times with replacement (**Bootstrap Sampling**) and train a separate model on each sample.

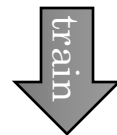
Original Dataset

x_1	y_1
x_2	y_2
x_3	y_3
x_4	y_4
x_5	y_5



Dataset 1

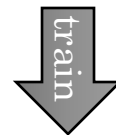
x_1	y_1
x_4	y_4
x_3	y_3
x_4	y_4
x_3	y_3



$$p_1(y | x)$$

Dataset 2

x_5	y_5
x_1	y_1
x_2	y_2
x_2	y_2
x_1	y_1



$$p_2(y | x)$$

Dataset 3

x_4	y_4
x_5	y_5
x_3	y_3
x_4	y_4
x_5	y_5



$$p_3(y | x)$$

Bagged Model:

$$p_{\text{BAG}}(y | x) = \frac{1}{3} \sum_{k=1}^3 p_k(y | x)$$

Not often applied in
Deep Learning.

Too expensive!

Dropout

- Inductive Biases
- Double Descent on the Bias-Variance Tradeoff
- Model Averaging
- Dropout
- Residual Connections
- Convolutional Neural Networks



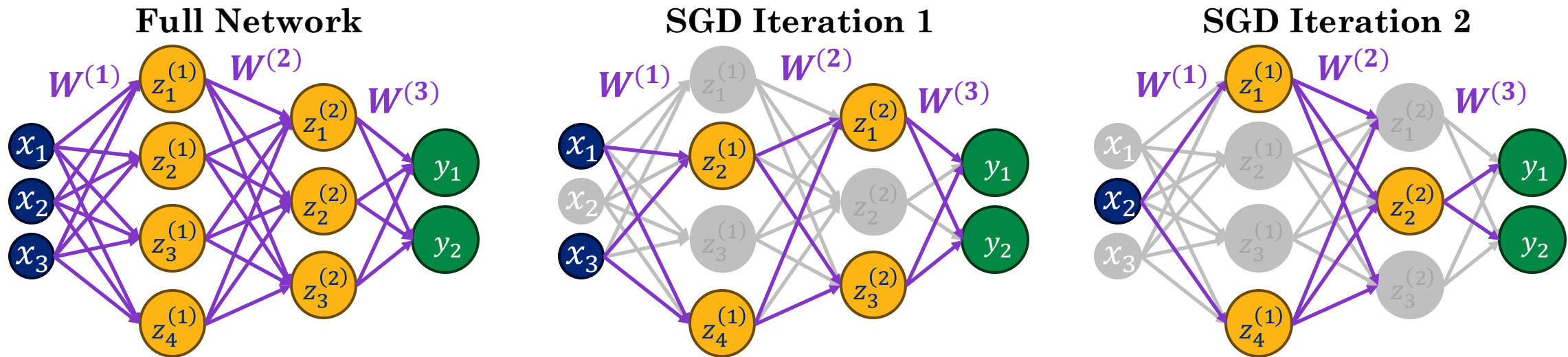
1030434



1030434

Dropout Regularization

Dropout **regularizes neural networks** by randomly “**disabling**” **neurons** during each iteration of SGD.



Implemented by sampling $r_m^{(l)} \sim \text{Bernoulli}(1 - \rho)$ and then using $\tilde{z}_m^{(l)} = z_m^{(l)} * r_m^{(l)}$ as the modified activations (typically $\rho \leq .5$).

At **test time** we set $r_m^{(l)} = 1 - \rho$ using all the activations (scaled).



What inductive bias is introduced by dropout?



Drop-out and Model Averaging

For a neural network with $M_\Sigma = D + \sum_{l=1}^L M_l$ total internal input neurons, drop-out **samples** from 2^{M_Σ} **possible “thinned” networks**.

During training, each step of SGD, we sample one of these thinned networks and apply a gradient updates to that network.

At test time, we are computing the **expected activation** by averaging over all possible combinations of input neurons.

Drop-out can be viewed as a form of **model averaging** over the 2^{M_Σ} thinned networks **with weight sharing**.

Implementing Drop-out in PyTorch



```
dropout = nn.Dropout(p=0.3)
```

```
dropout.train()
```

```
display(dropout(torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]])))
```

```
dropout.eval()
```

```
display(dropout(torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]])))
```

✓ 0.0s

```
tensor([[ 1.4286,  2.8571,  0.0000],  
        [ 5.7143,  7.1429,  0.0000],  
        [10.0000, 11.4286, 12.8571]])
```

```
tensor([[1., 2., 3.],  
        [4., 5., 6.],  
        [7., 8., 9.]])
```

PyTorch uses a different (but equivalent) implementation in which the **sampled activations are scaled up** by $\frac{1}{1-\rho}$ during training and then **un-altered** at test time.

Residual Connections

- Inductive Biases
- Double Descent on the Bias-Variance Tradeoff
- Model Averaging
- Dropout
- Residual Connections
- Convolutional Neural Networks



1030434



Vanishing Gradients in Deep Networks

Training deep networks with **hundreds of layers** is challenging.

- **Vanishing Gradients:** error signals get multiplicatively diluted across layers
- **Shattered Gradients:** many stacked non-linearities result in a rough loss surface.

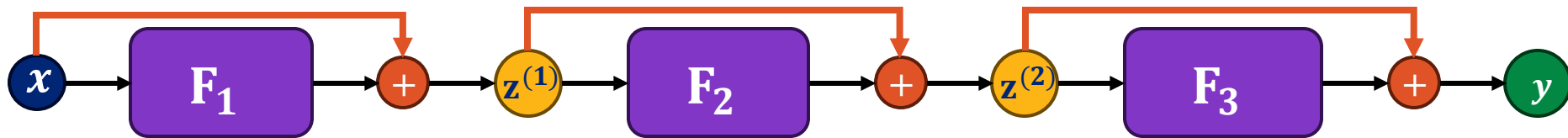
Need a way to **connect lower layers** in the network directly to the predictions (and subsequent loss).



1030434

Residual Networks

Residual networks enable the training of very deep networks (e.g., hundreds of layers) by introducing **residual connections** (skip connections) **around blocks** (sub-networks F_l).



We can express a residual block as:

$$\mathbf{z}^{(l)} = F_l(\mathbf{z}^{(l-1)}) + \mathbf{z}^{(l-1)}$$

Example:

$$F_l(\mathbf{z}) = \mathbf{W}^{(l)} \text{ReLU}(\mathbf{z})$$

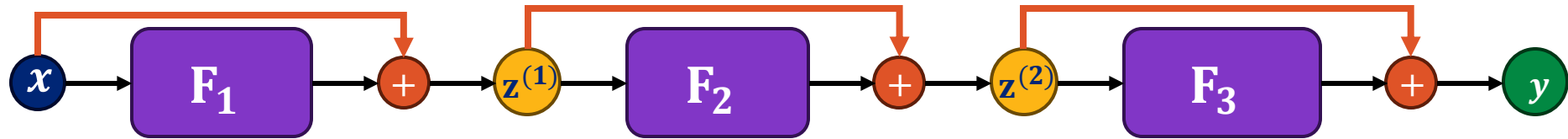
The term **residual** comes from the fact that F_l is modeling the **residual** between the **output** and the **input** of the layer:

$$F_l(\mathbf{z}^{(l-1)}) = \mathbf{z}^{(l)} - \mathbf{z}^{(l-1)}$$



Residual Networks as Ensembles

Residual networks can be viewed as an **ensemble of networks with varying depths**:



If we expand our example network to get:

$$y = F_3(F_2(F_1(\mathbf{x}) + \mathbf{x}) + \mathbf{z}^{(1)}) + \mathbf{z}^{(2)}$$

Substituting the $\mathbf{z}^{(1)} = F_1(\mathbf{x}) + \mathbf{x}$ and $\mathbf{z}^{(2)} = F_2(F_1(\mathbf{x}) + \mathbf{x}) + F_1(\mathbf{x}) + \mathbf{x}$:

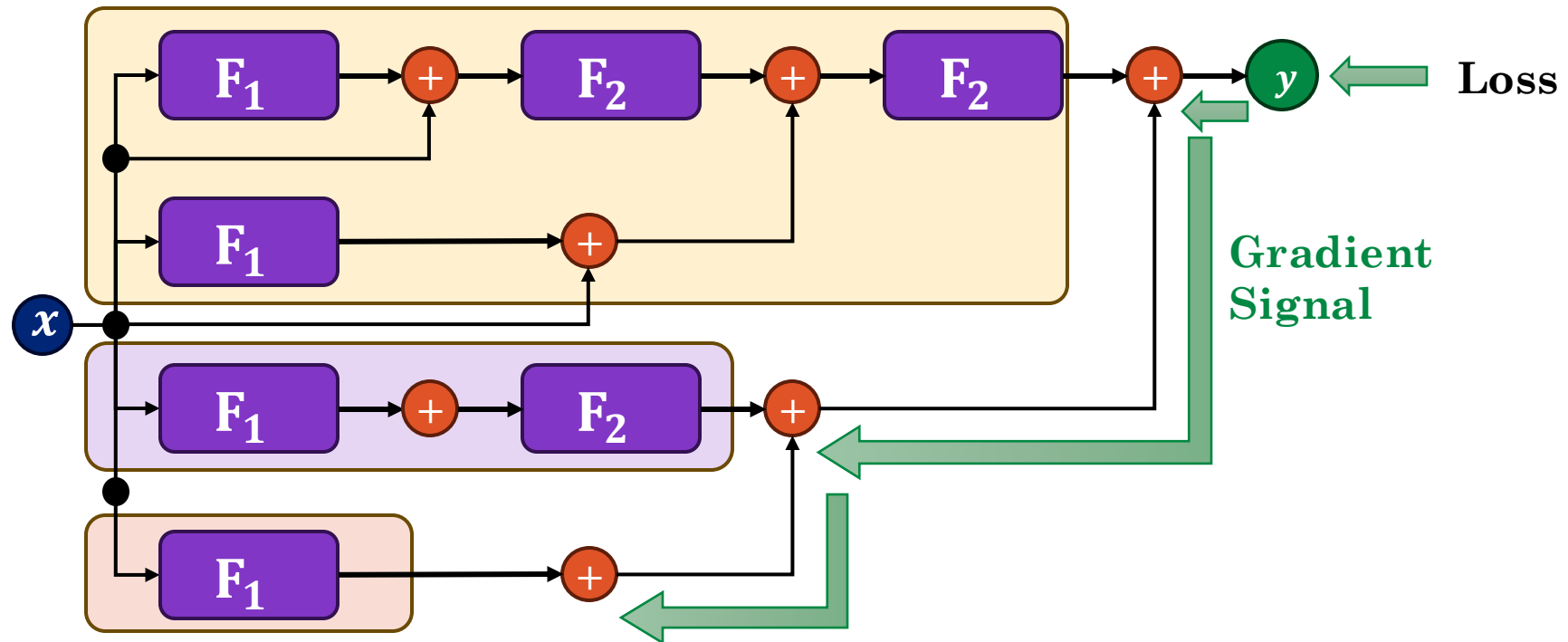
$$y = \underbrace{F_3(F_2(F_1(\mathbf{x}) + \mathbf{x}) + F_1(\mathbf{x}) + \mathbf{x})}_{\text{Large Network}} + \underbrace{F_2(F_1(\mathbf{x}) + \mathbf{x})}_{\text{Medium Network}} + \underbrace{F_1(\mathbf{x}) + \mathbf{x}}_{\text{Small Network}}$$

Expanding the Residual Network Graph



The **expanded ensemble structure** of the residual network shows how **input layers** receive more **direct gradient signal**.

$$y = \boxed{F_3(F_2(F_1(\mathbf{x}) + \mathbf{x}) + F_1(\mathbf{x}) + \mathbf{x})} + \boxed{F_2(F_1(\mathbf{x}) + \mathbf{x})} + \boxed{F_1(\mathbf{x}) + \mathbf{x}}$$





1030434

Demo

Probably the coolest demo yet.

Model Type: MLP

Layers: 5

Neurons/La...: 10

Activation: tanh

Normalizati...: none

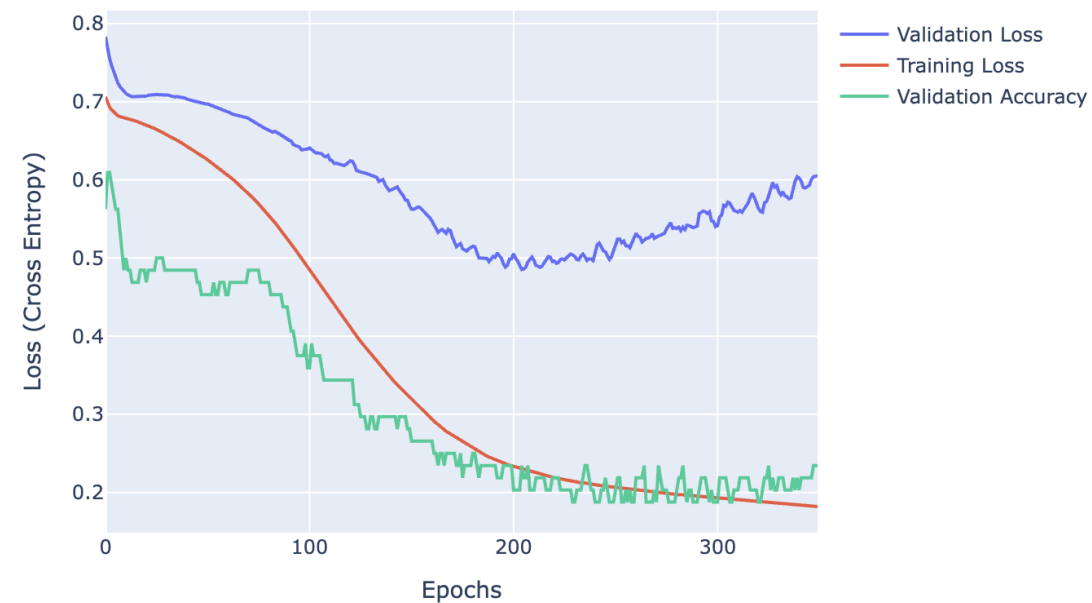
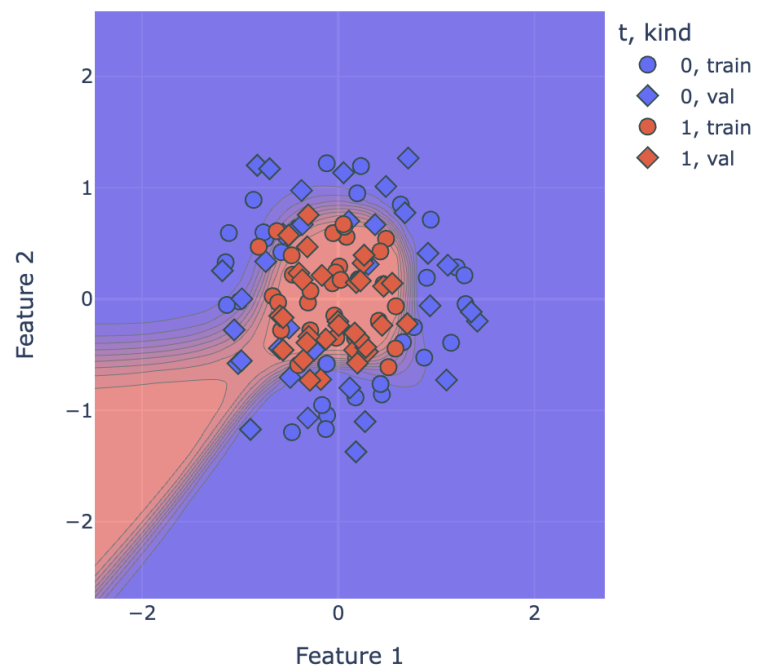
Dropout: 0.00

Learning R...: 0.00

Weight De...: 0.00

Batch Size: 32

Epochs: 350



Convolutional Neural Networks

- Inductive Biases
- Double Descent on the Bias-Variance Tradeoff
- Model Averaging
- Dropout
- Residual Connections
- Convolutional Neural Networks



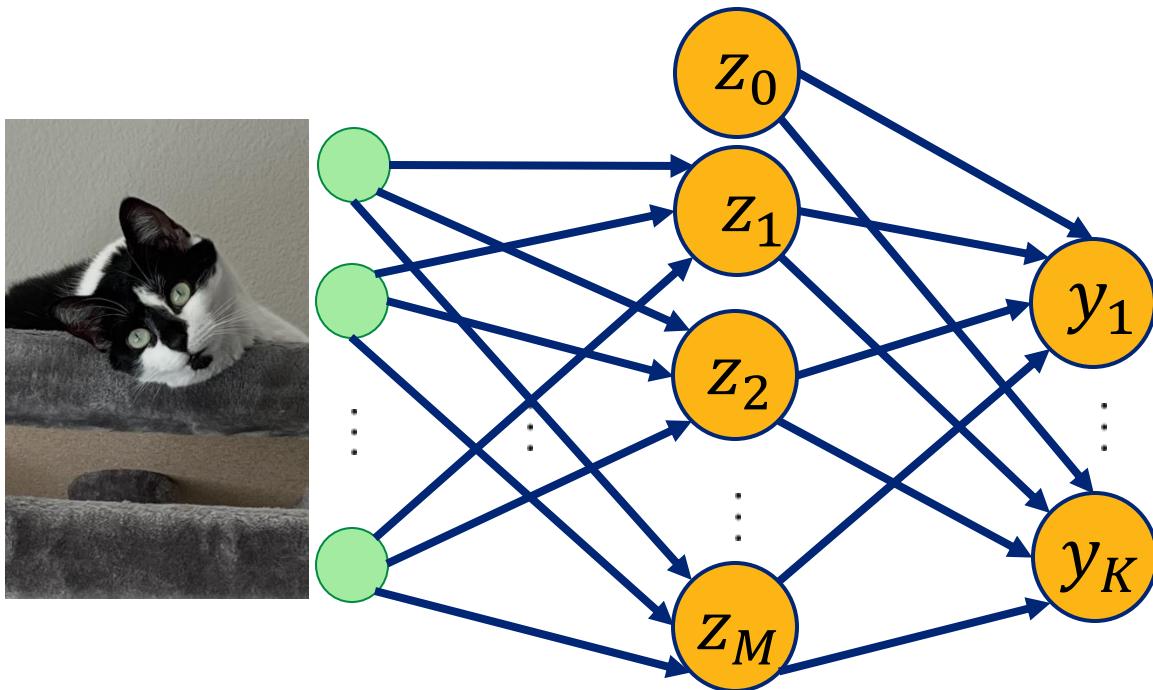
1030434

Story So Far



1030434

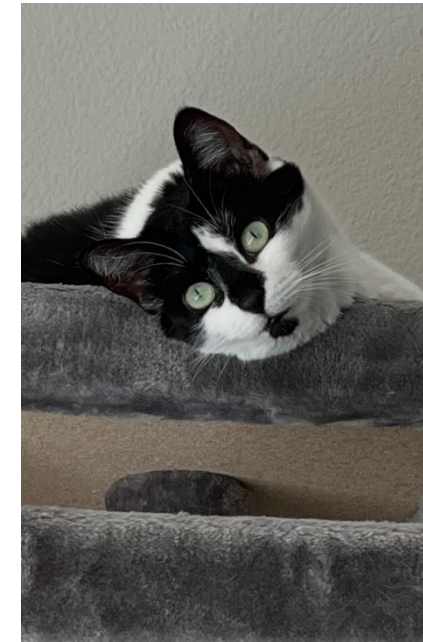
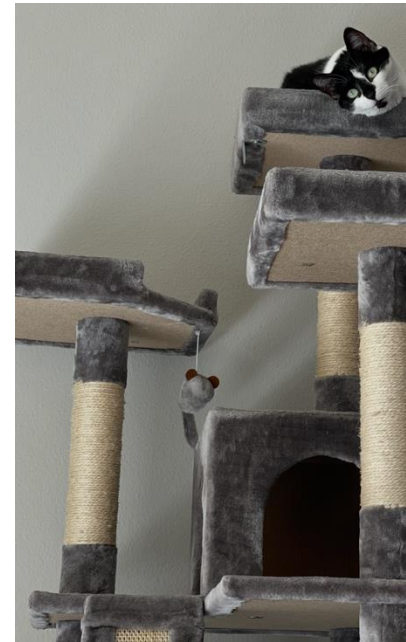
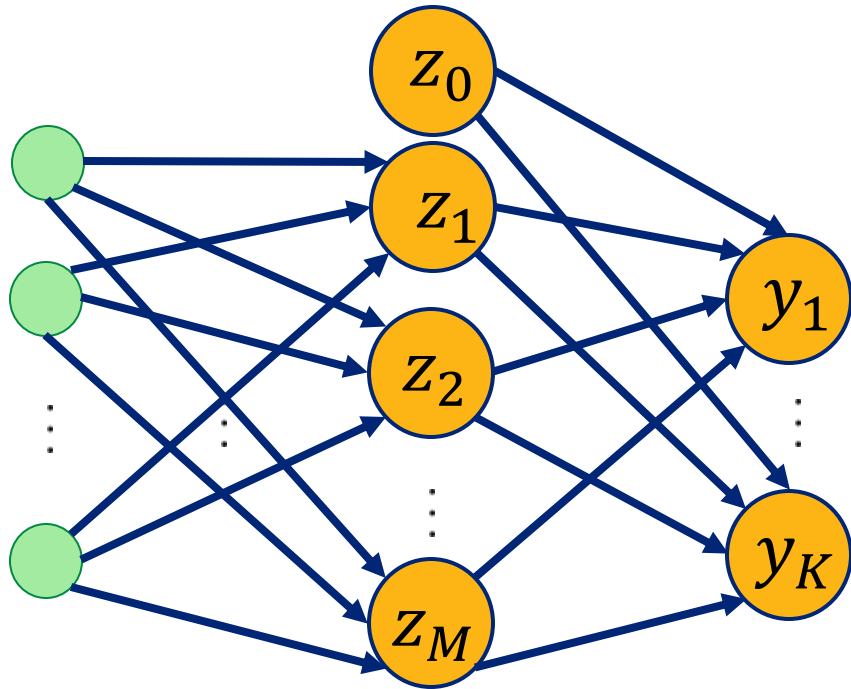
- NNs are universal function approximators.
- NNs can be trained through variations of gradient descent.
- Can recognize patterns in data.



A Problem



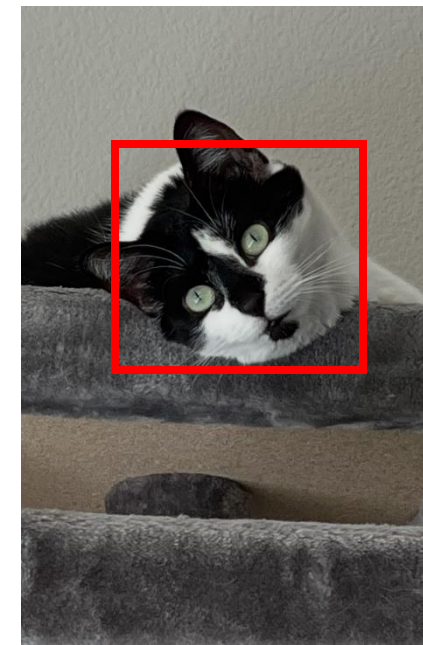
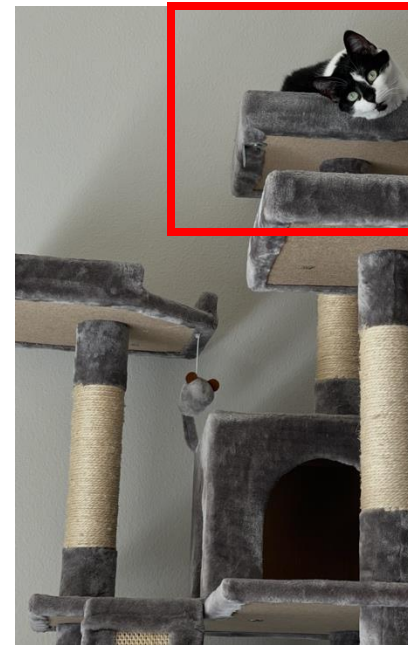
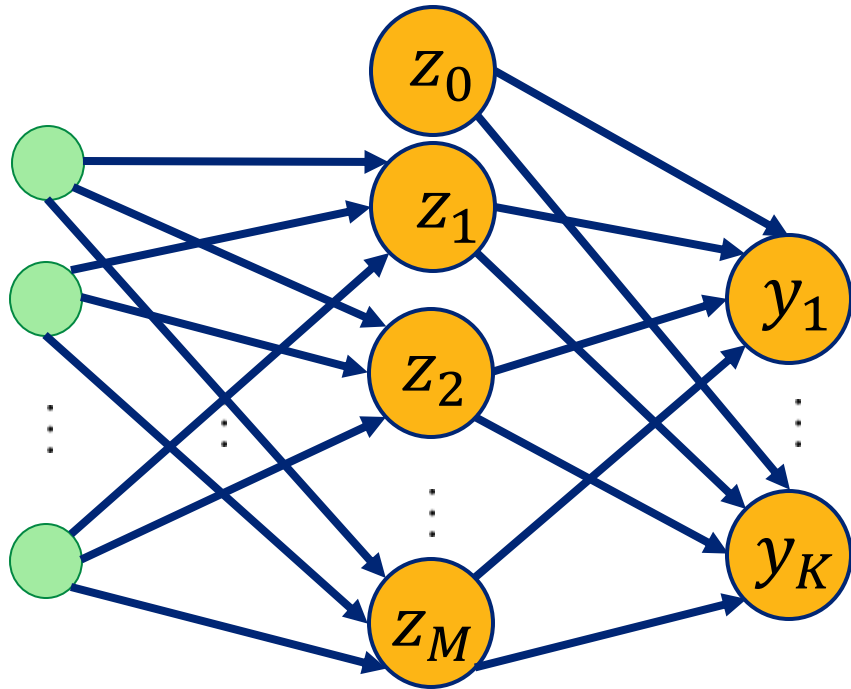
- Will a NN that recognizes the left image as a cat also recognize the one on the right one as a cat?



A Problem



- Will a NN that recognizes the left image as a cat also recognize the one on the right one as a cat?



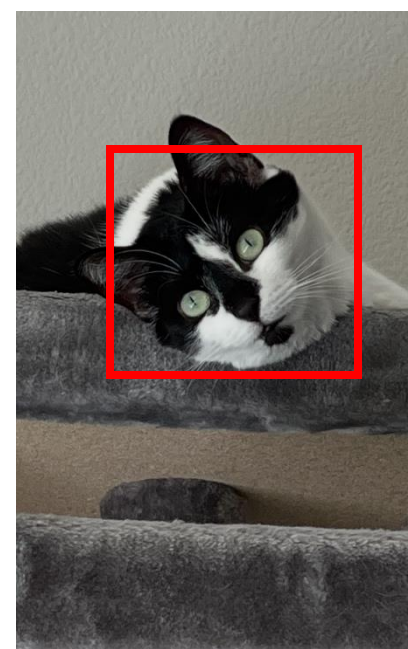
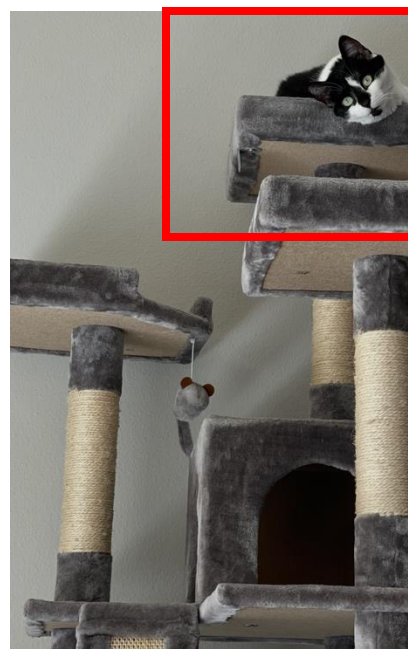
A network that recognizes "cat" feature regardless of the location and scale of the target object.

A Problem



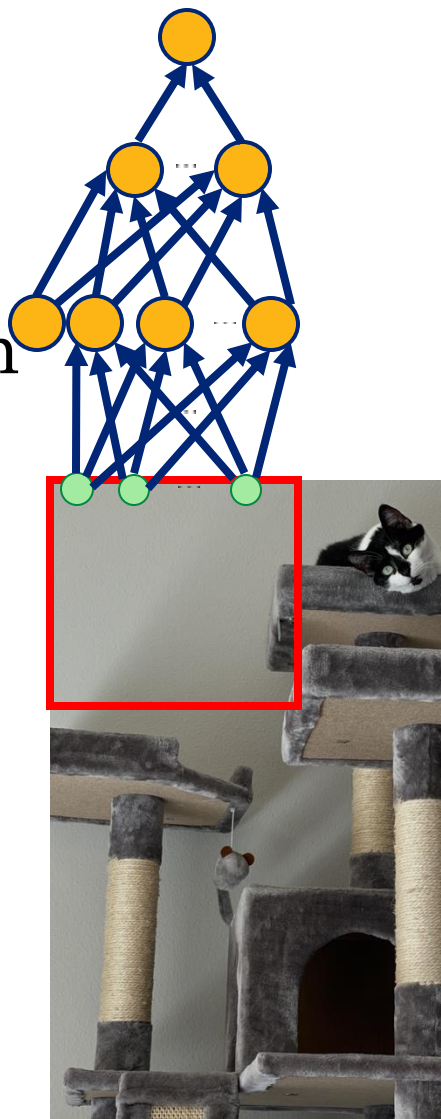
1030434

**Network must be shift
invariant.**



Solution: Scanning

- Scan for the presence of the cat in the image.

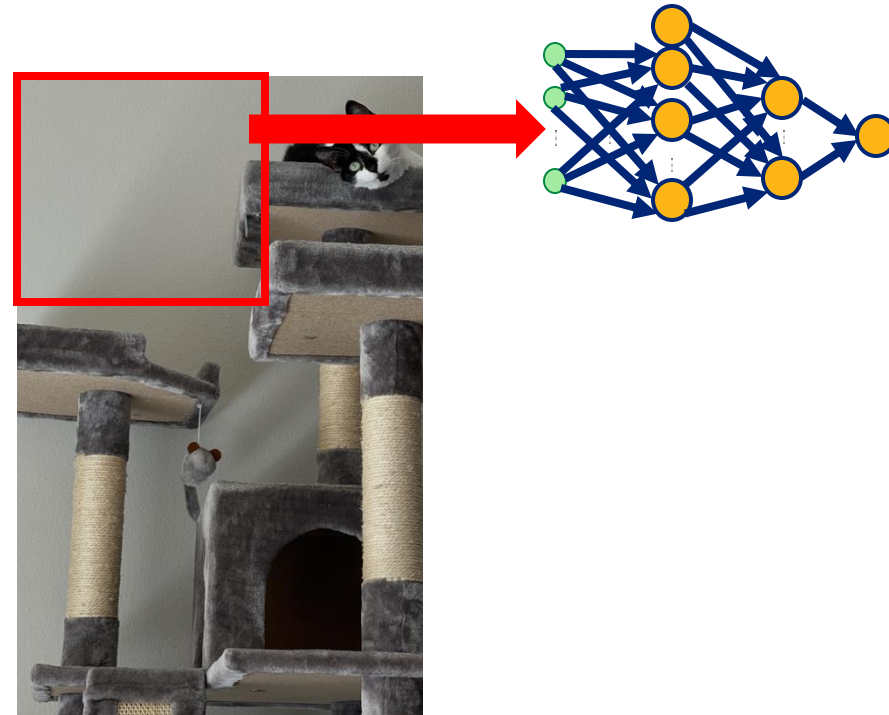


1030434

Scanning Is Referred to as Convolution



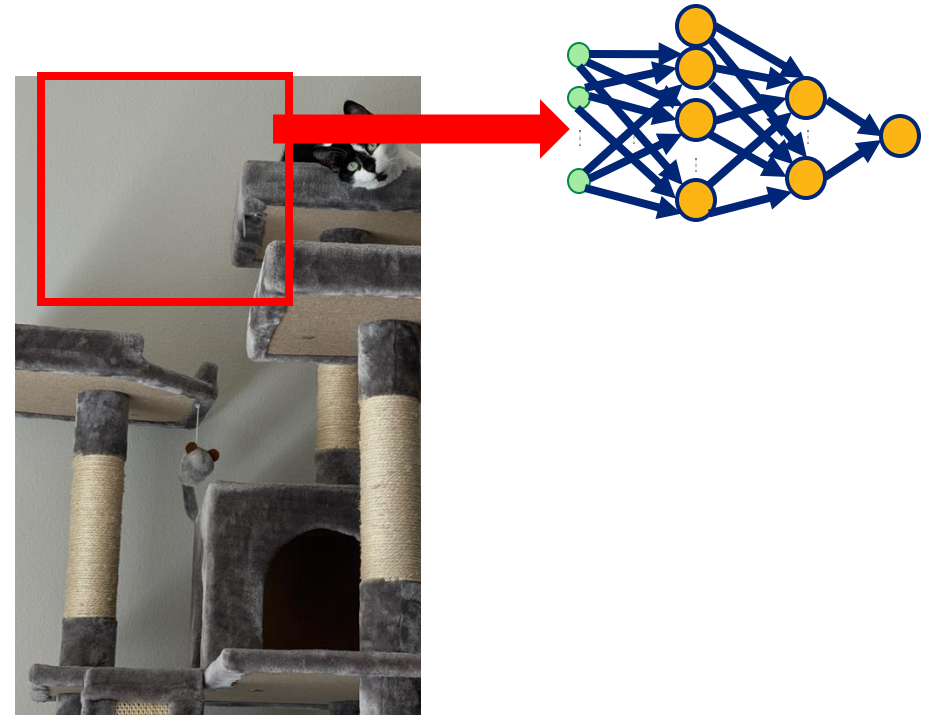
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



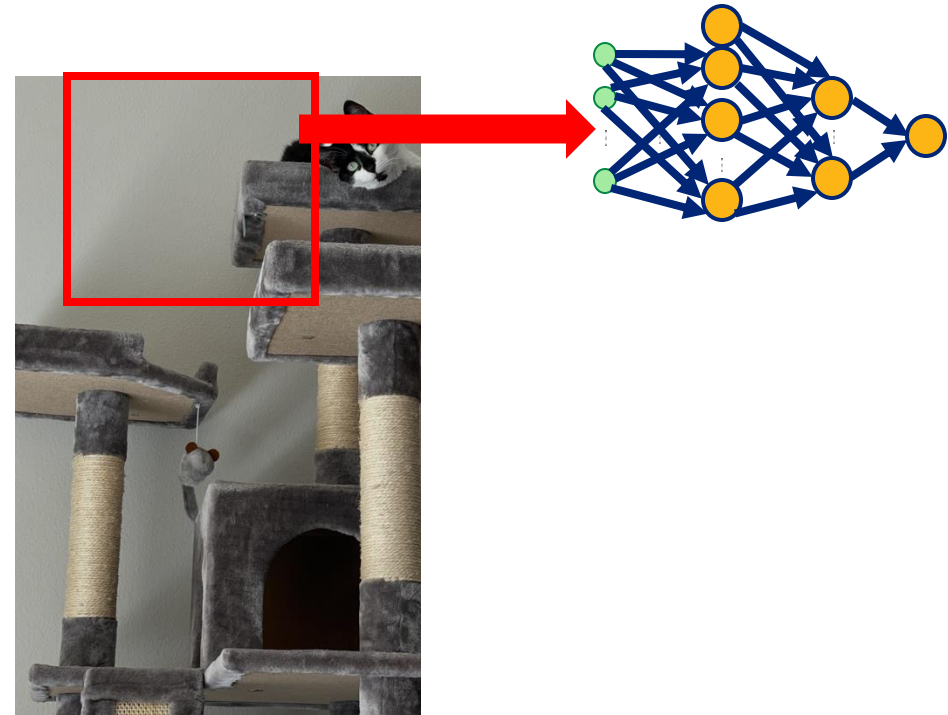
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



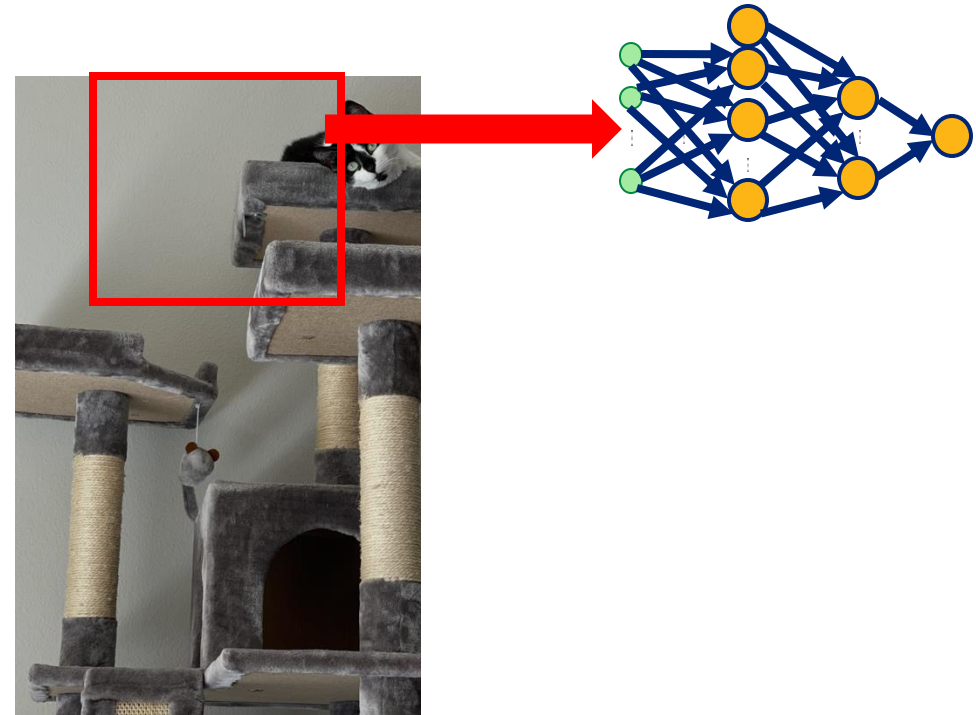
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



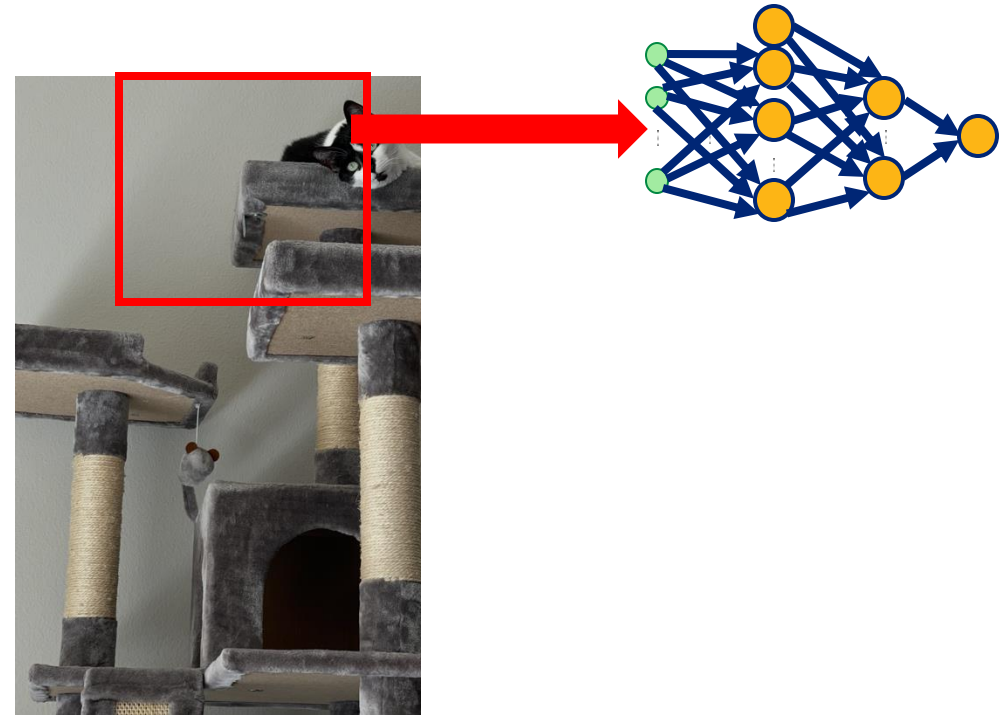
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



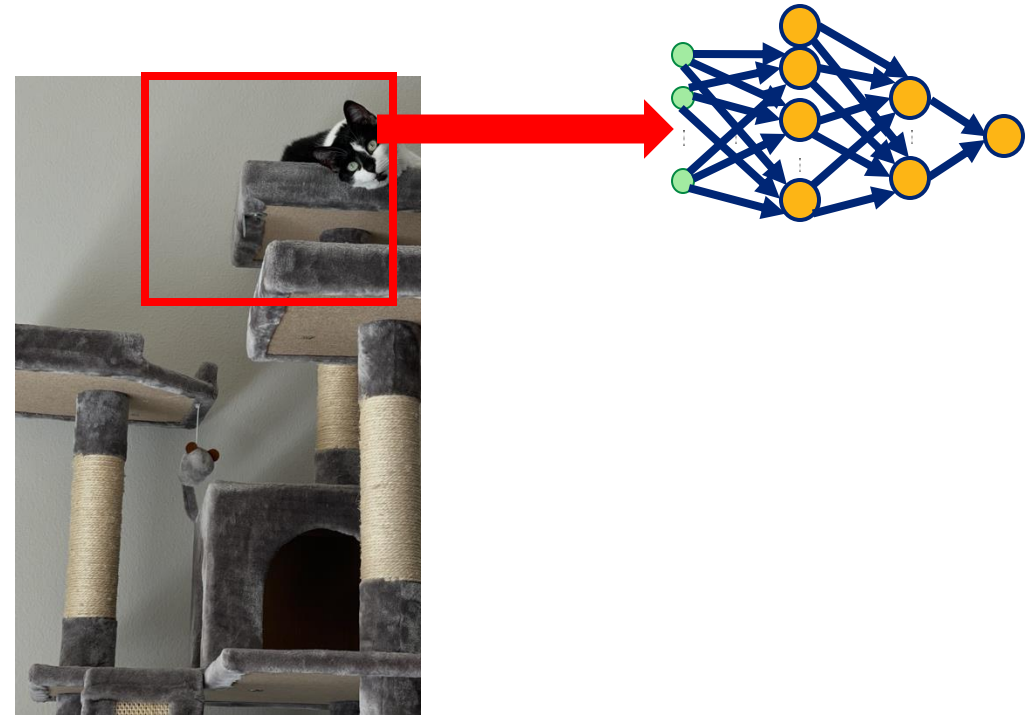
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



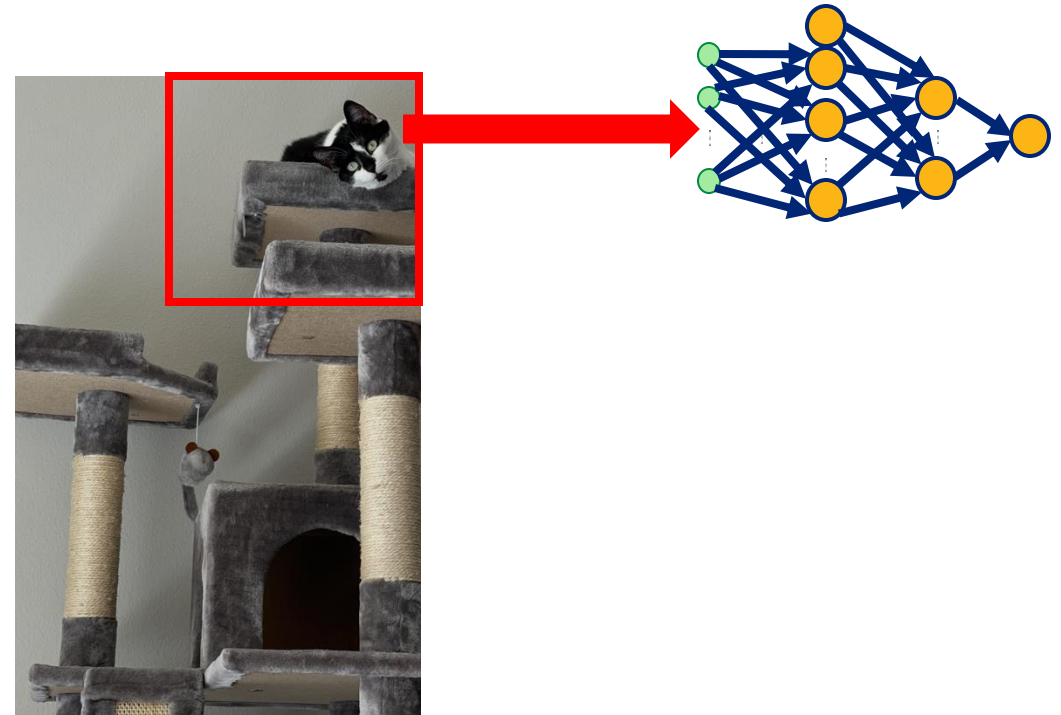
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



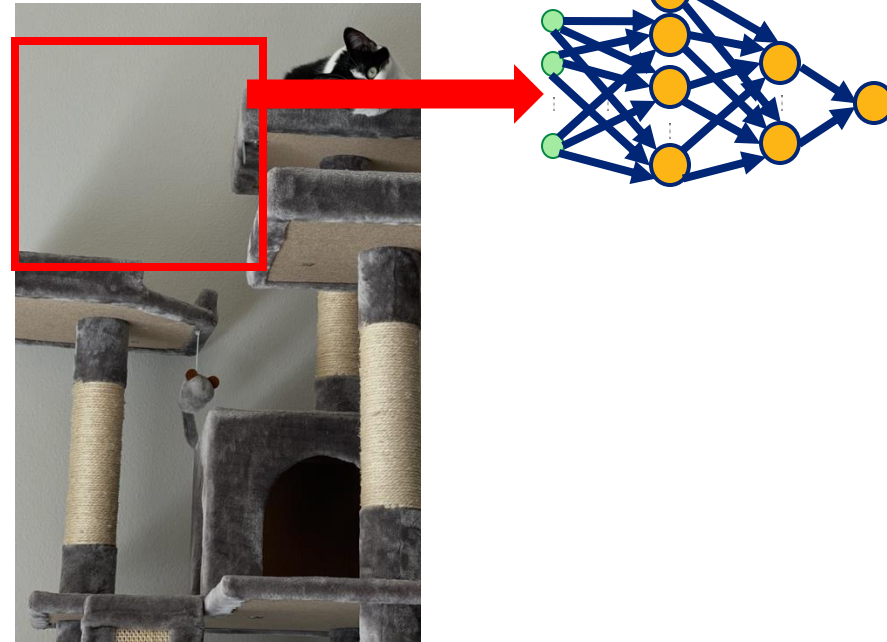
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



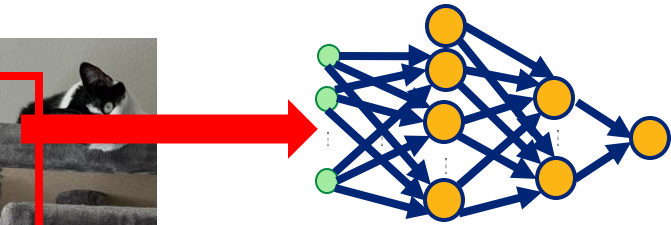
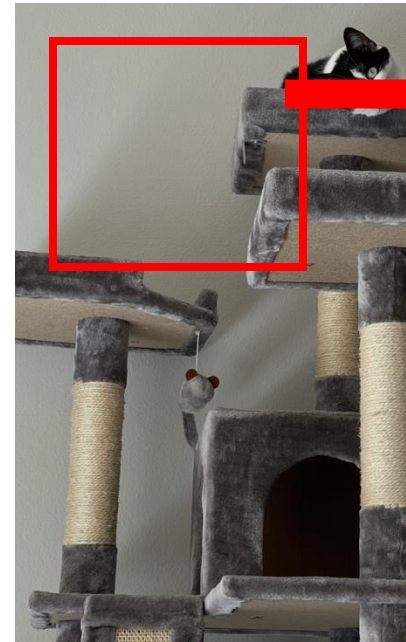
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



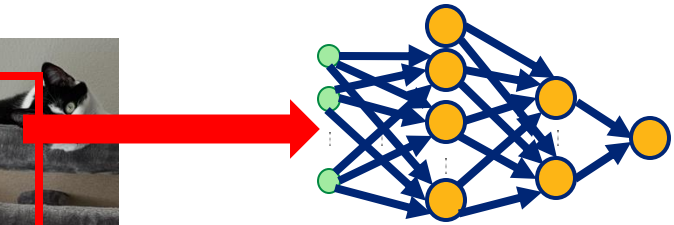
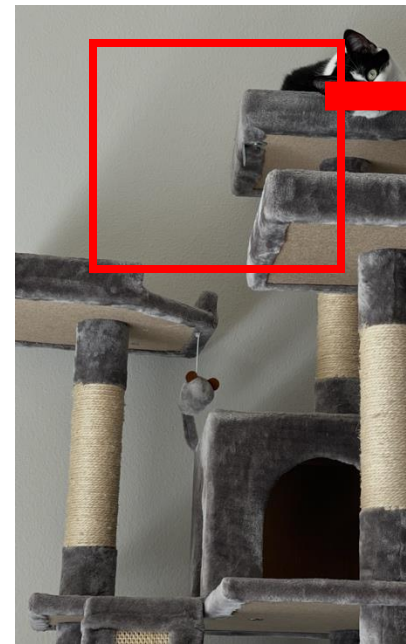
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



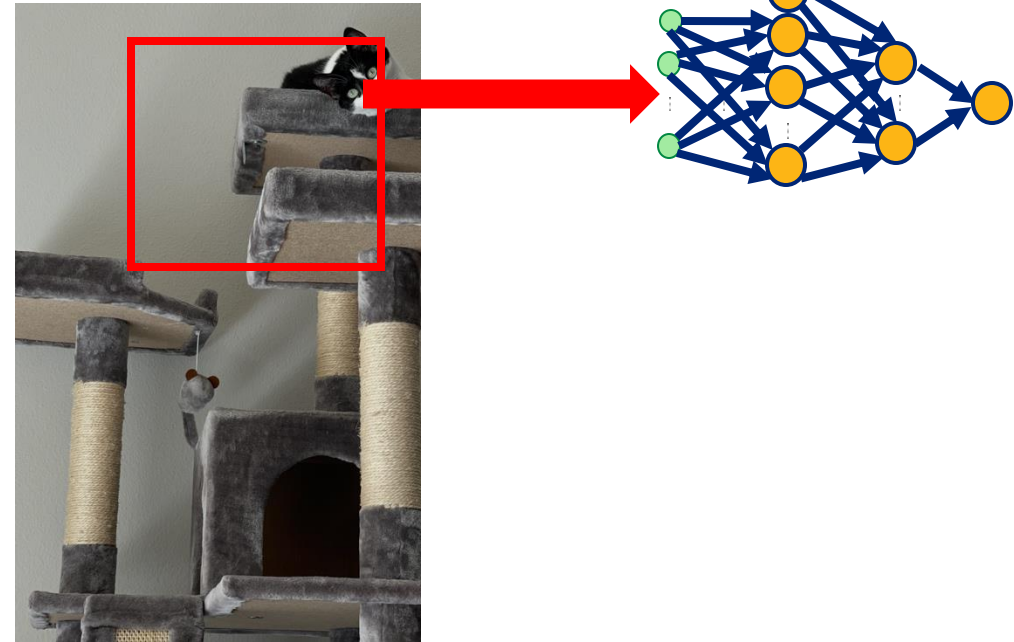
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



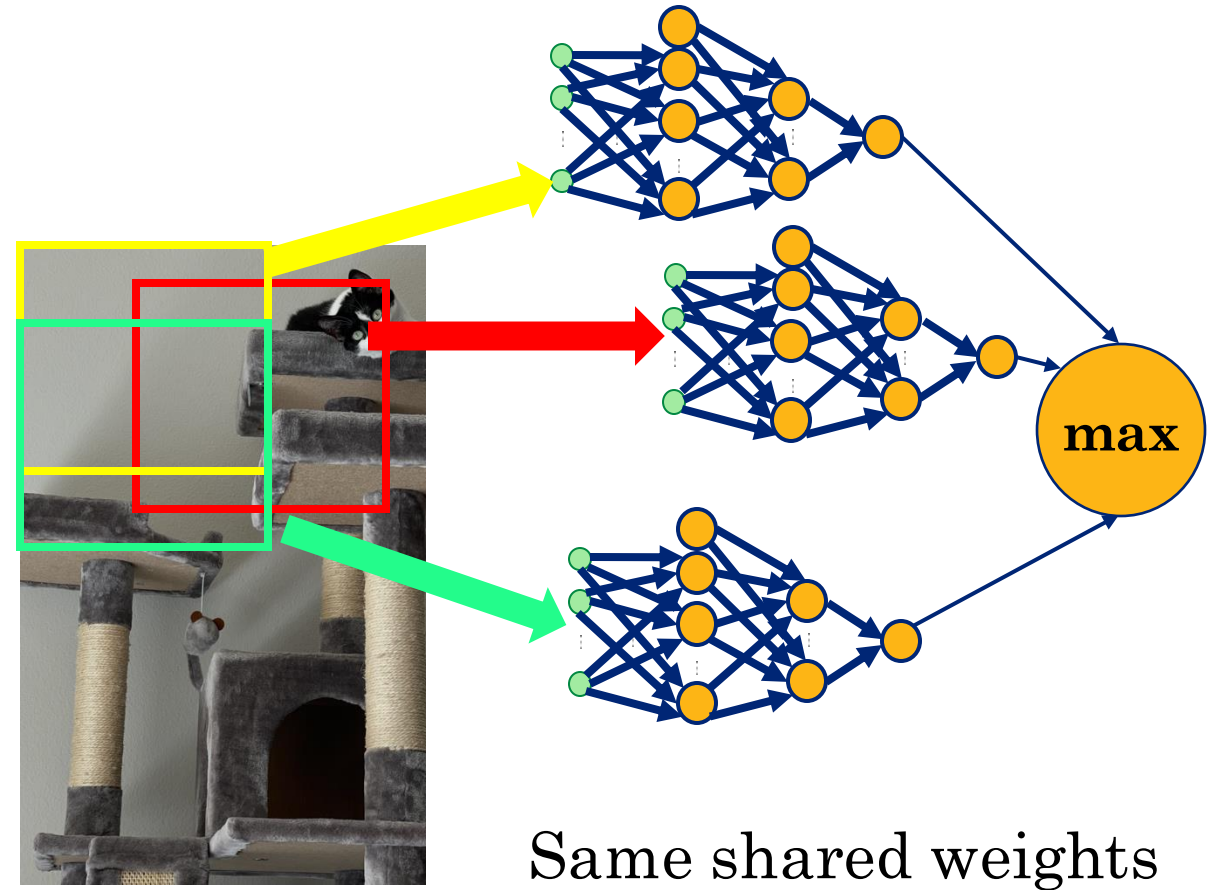
1030434

- We need to apply a feature extractor in each region of the image.

Maximum of all the outputs
(Equivalent of Boolean OR)



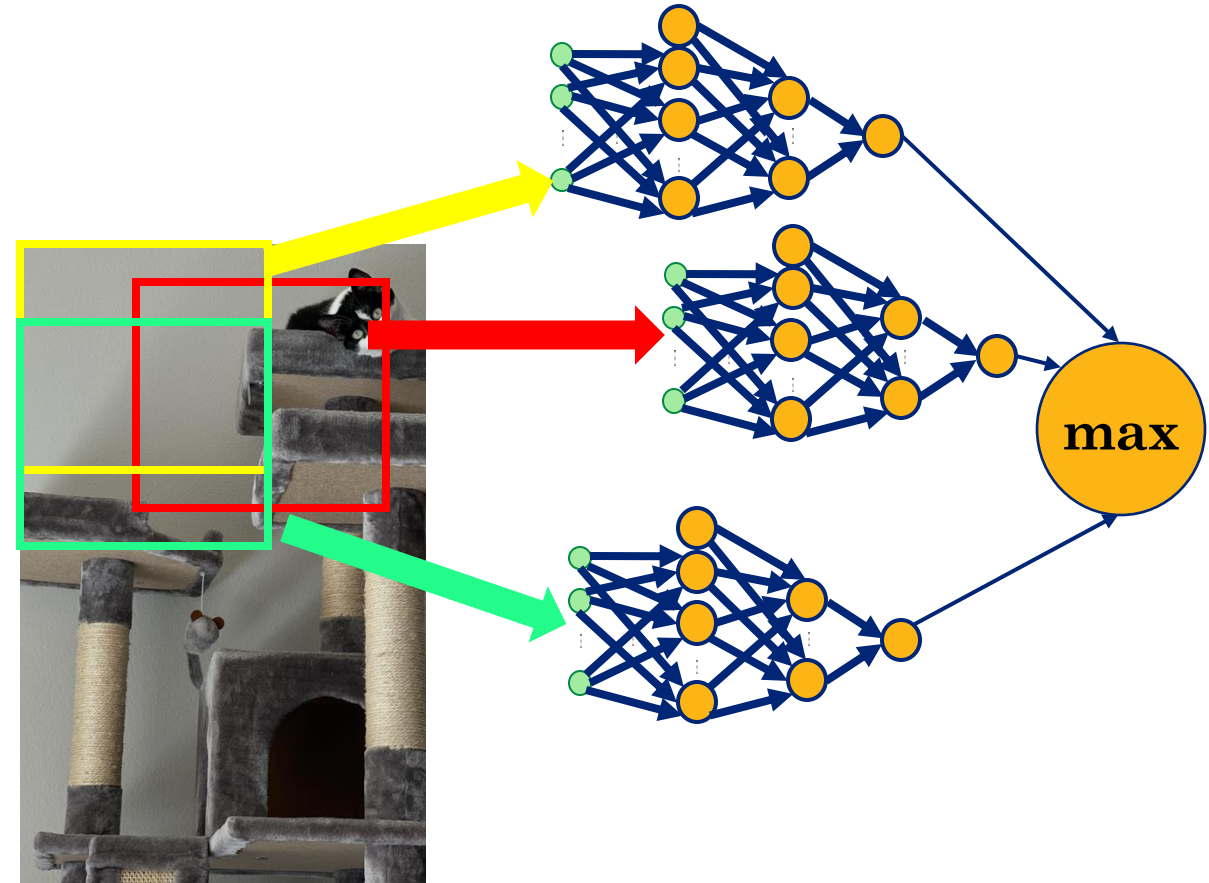
The strength of having the
feature anywhere in the image



Scanning Is Referred to as Convolution



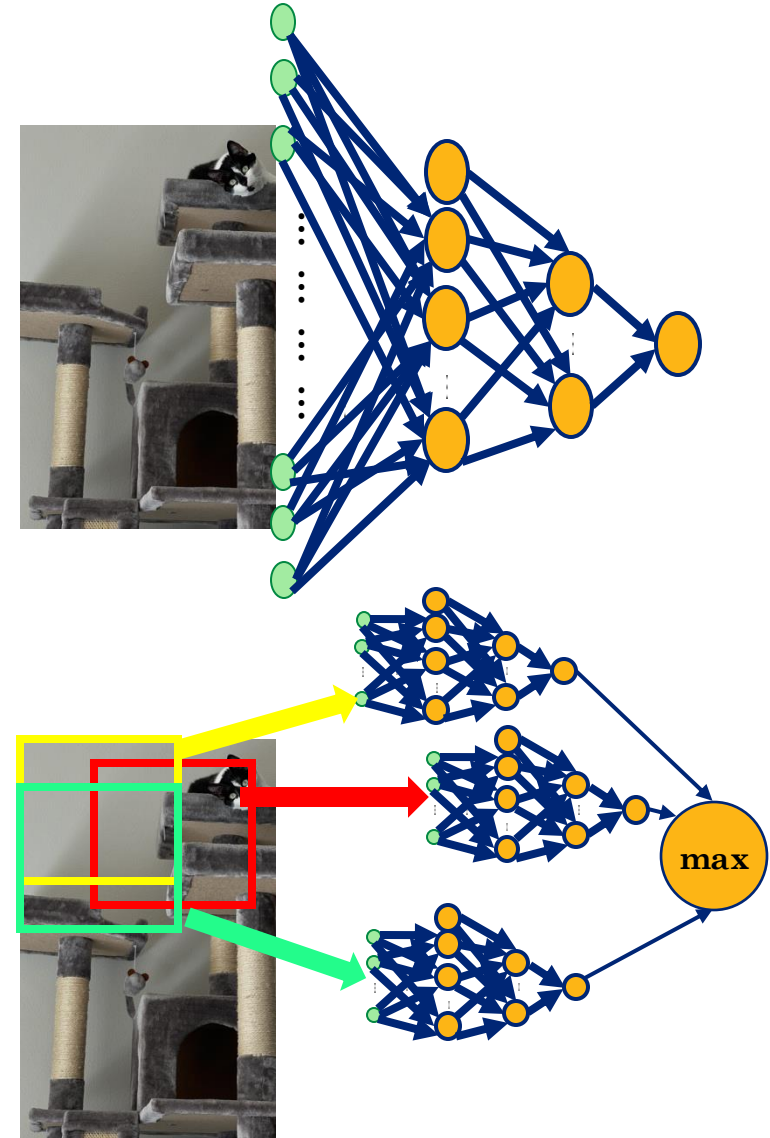
- The entire operation can be viewed as one giant network.
 - With many subnetworks, one per window.
- Restriction: All subnetworks are identical.
- The network is shift-invariant!



Regular Network vs. Scanning Network



- In a FC-NN, each neuron is connected by a unique weight to every neuron in the previous layer.
 - Entries in the weight matrix are unique.
- In a scanning NN, each neuron is connected to a subset of neurons in the previous layer
 - The weights matrix is sparse.
 - The weights matrix is block structured with identical blocks.



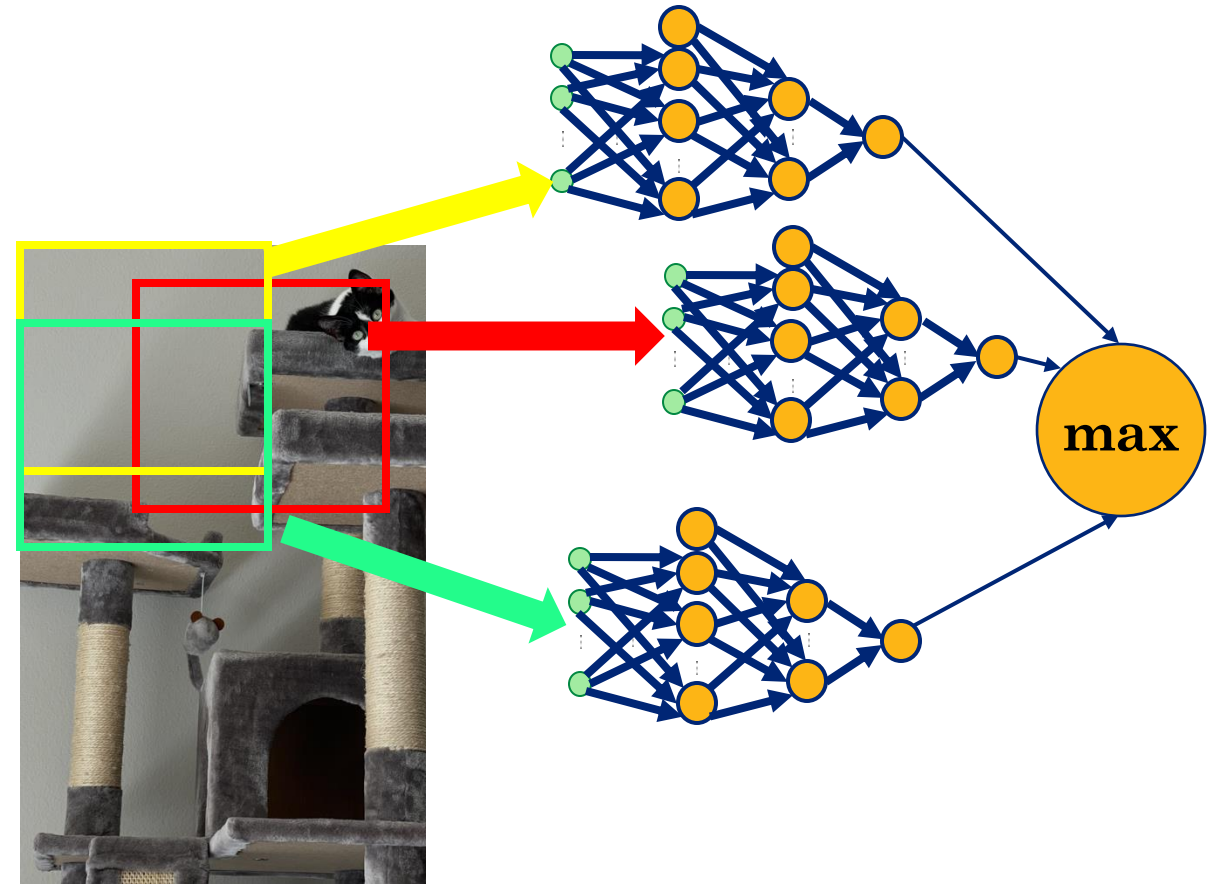


1030434

Back to Scanning Network

- Patches are referred to as "receptive fields".
- The weights form a small two-dimensional grid known as a filter or a kernel.
- Assume we unroll the 2D structures into a vector: output from each receptive field is calculated as:

$$z = \text{ReLU}(\mathbf{w}^T \mathbf{x} + w_0)$$

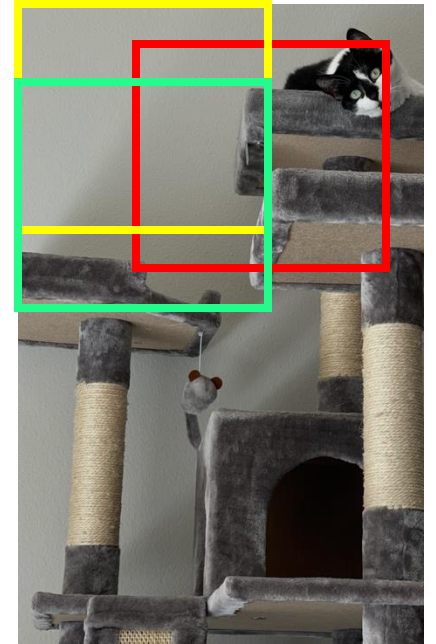


How to Identify a Feature in a Receptive Field?



$$z = \text{ReLU}(\mathbf{w}^T \mathbf{x} + w_0)$$

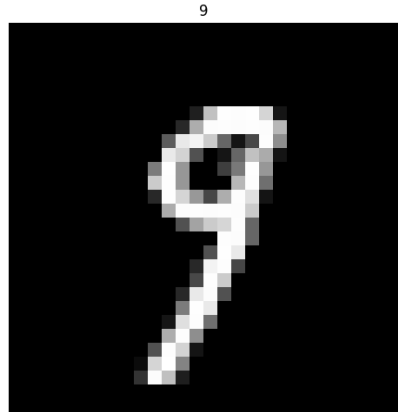
- Suppose that $\mathbf{w}$ and w_0 are fixed and we want to identify $\mathbf{x}$ with the largest output response. Let's also assume that norm $\|\mathbf{x}\|$ is fixed.
- The solution that maximizes $\mathbf{w}^T \mathbf{x}$, and the response of the hidden unit, is of the form $\mathbf{x} = \alpha \mathbf{w}$ for some coefficient α .
- The maximum output response from hidden unit occurs when it detects a patch of image that, up to an overall scaling, looks like the kernel image.



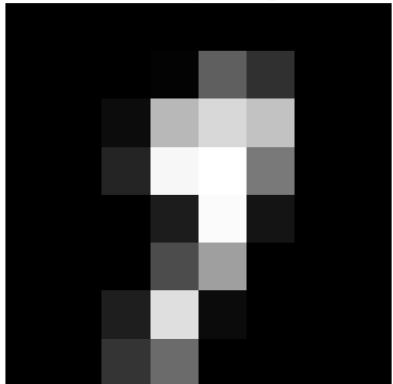
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector

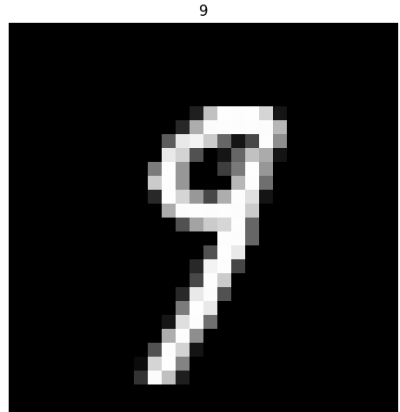
$\odot$

-1

How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image

Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector

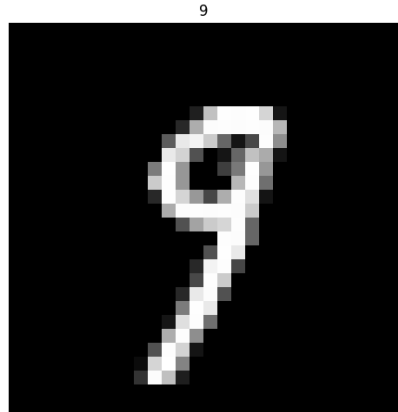
$\odot$

-1 0

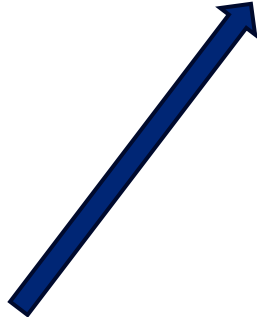
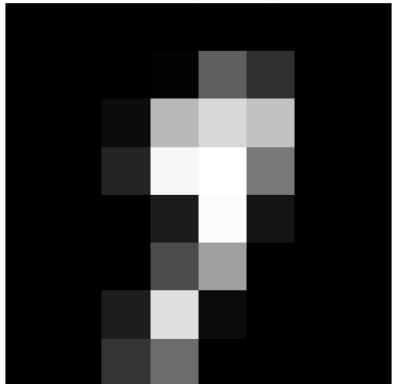
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

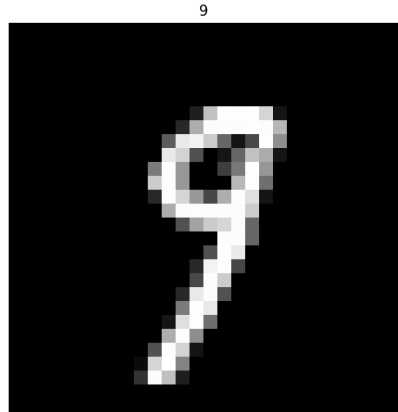
-1 0 1

Horizontal line detector

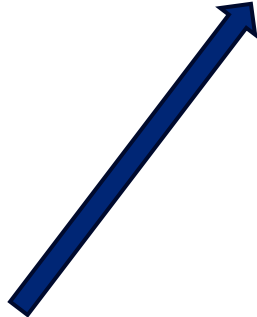
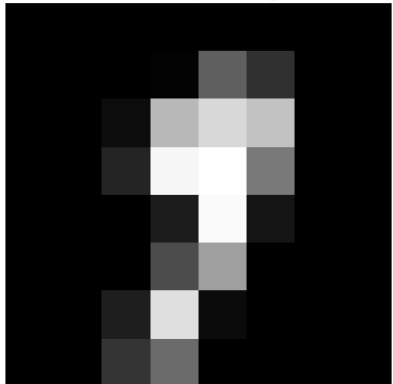
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

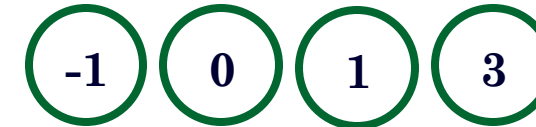
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

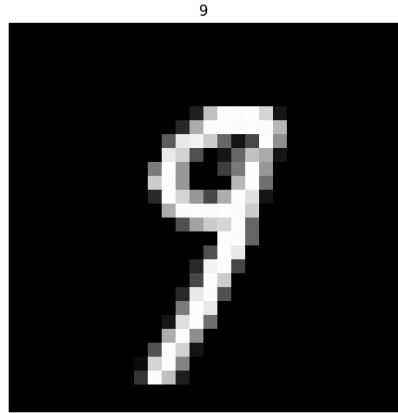
Horizontal line detector



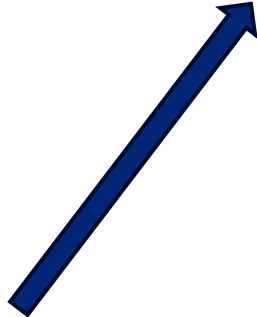
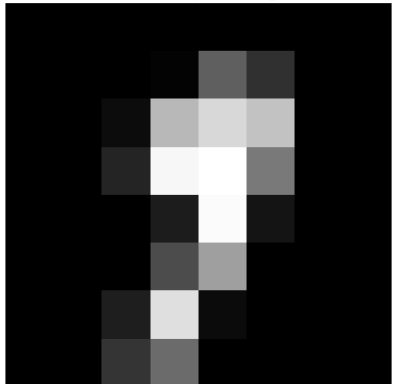
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

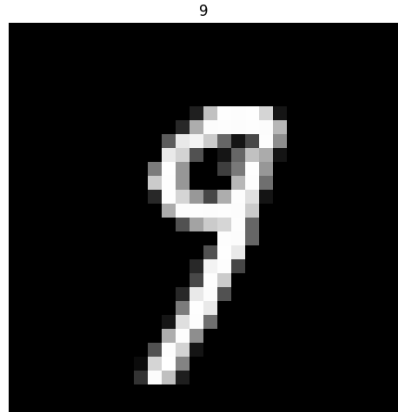
Horizontal line
detector



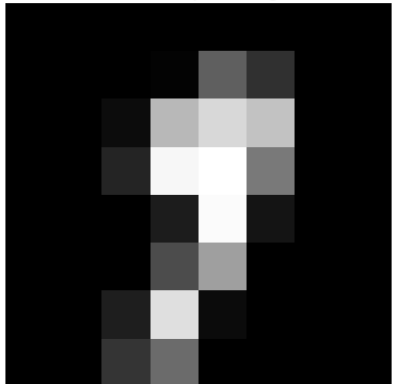
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

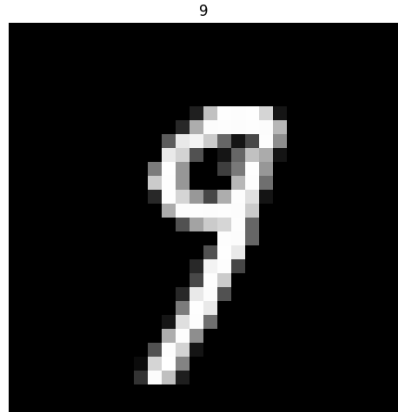
Horizontal line detector



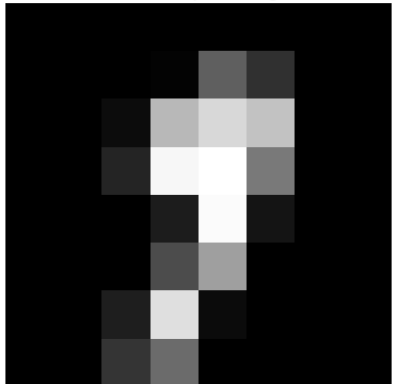
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

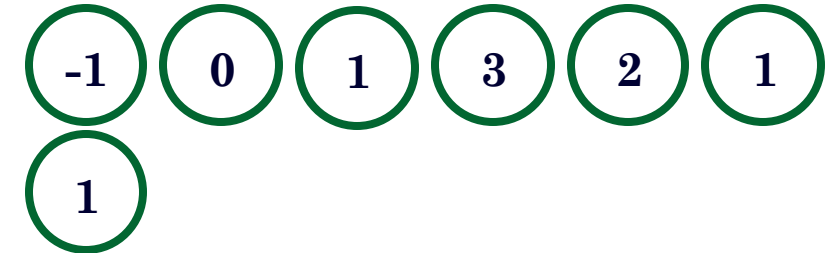
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

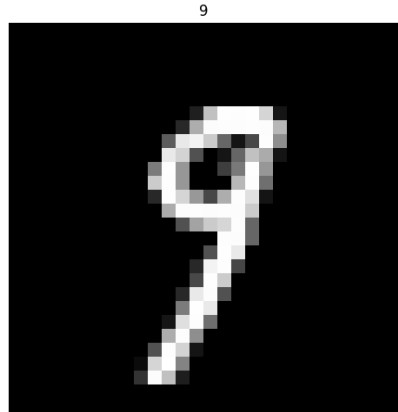
Horizontal line detector



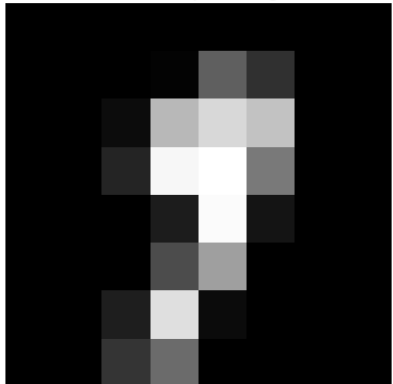
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

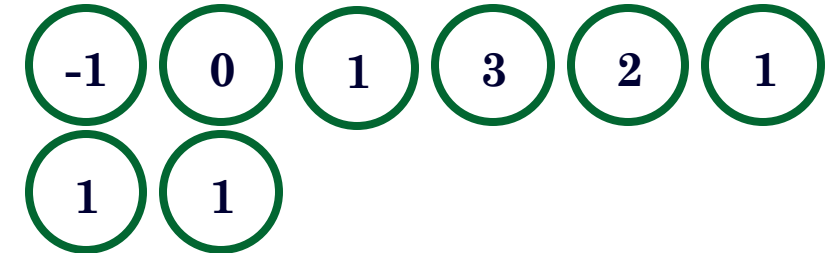
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

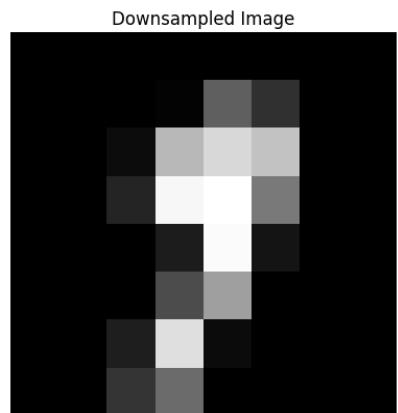
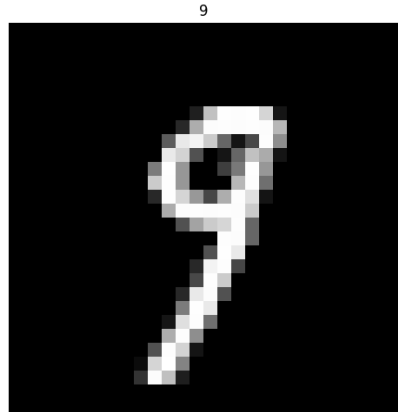
Horizontal line detector



How to Implement Scanning of Features with Weight-Sharing?



1030434



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

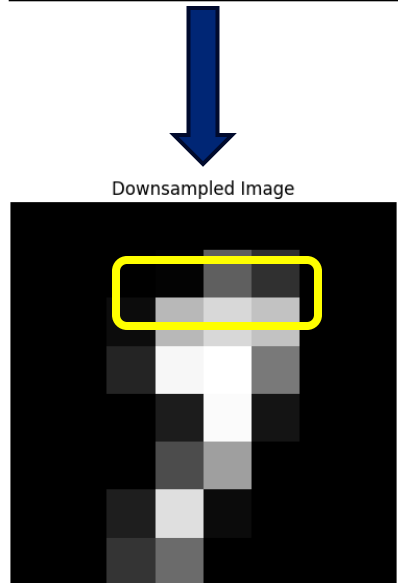
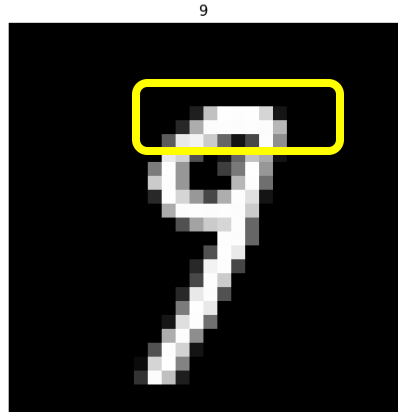
Horizontal line detector

-1	0	1	3	2	1
1	1	1	0	0	0
1	1	1	0	0	0
-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1
1	1	2	1	1	0

How to Implement Scanning of Features with Weight-Sharing?



1030434



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

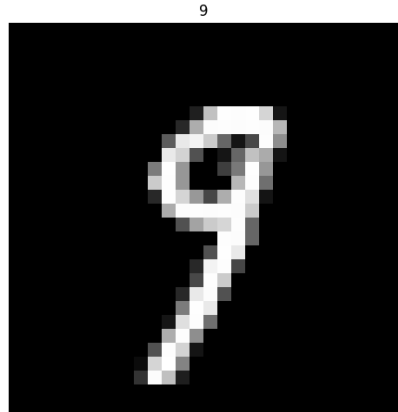
Horizontal line detector

-1	0	1	3	2	1
1	1	1	0	0	0
1	1	1	0	0	0
-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1
1	1	2	1	1	0

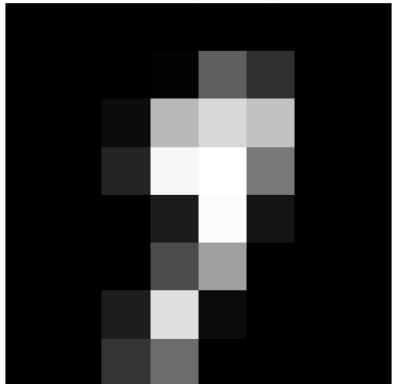
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

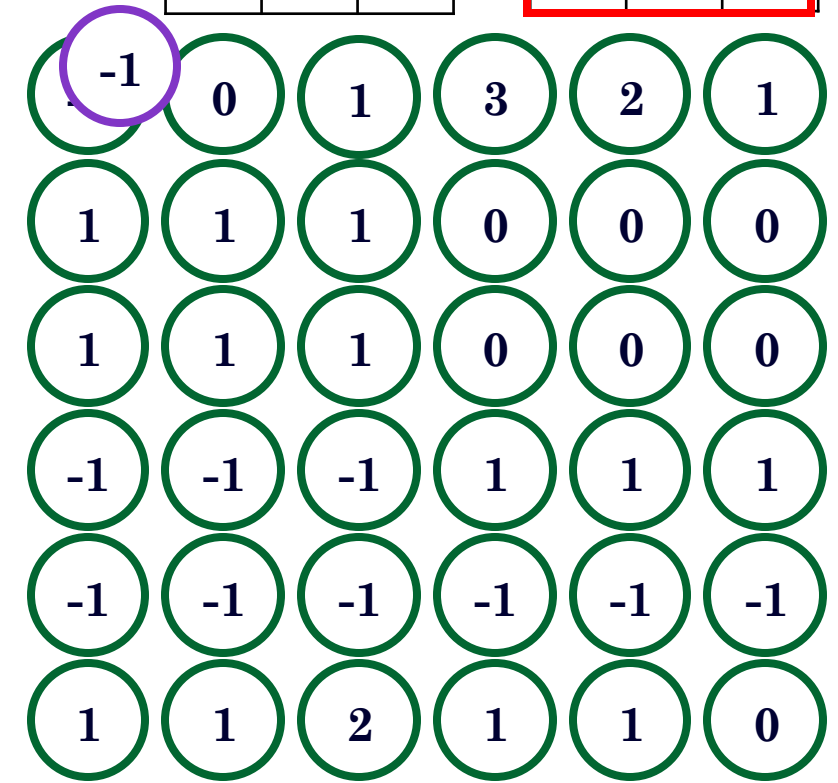
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

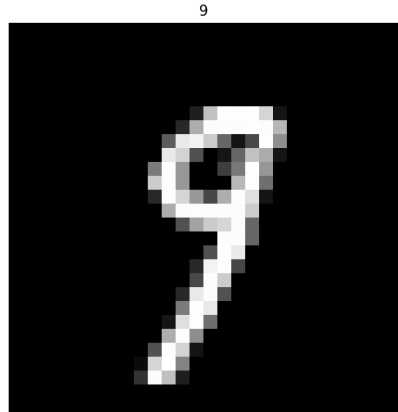
$\odot$



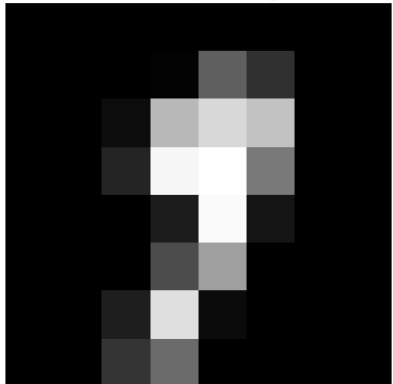
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

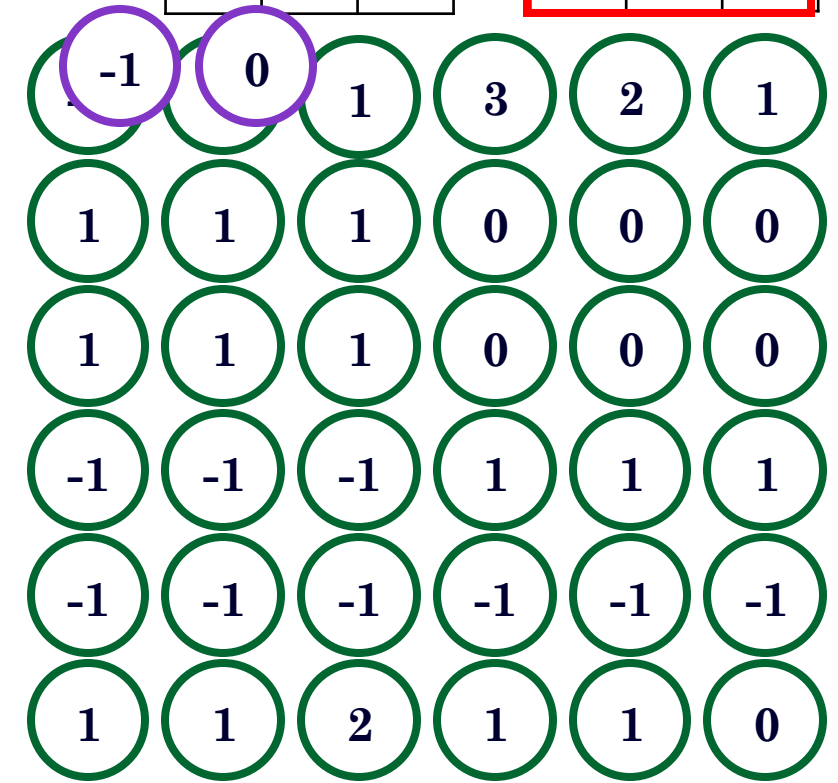
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

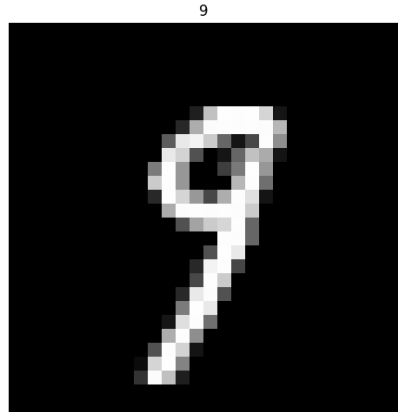
$\odot$



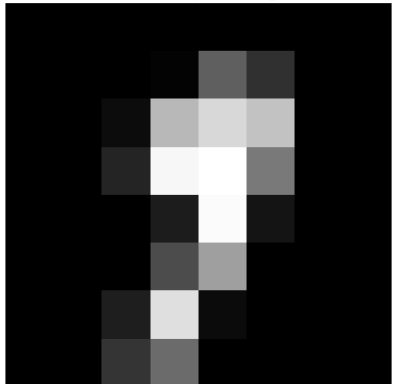
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

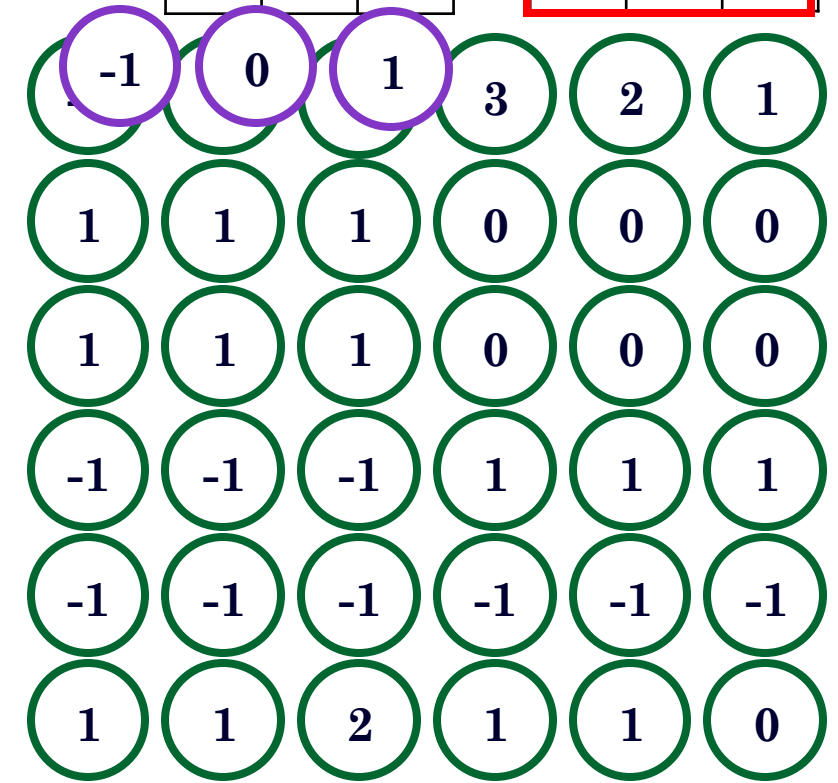
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

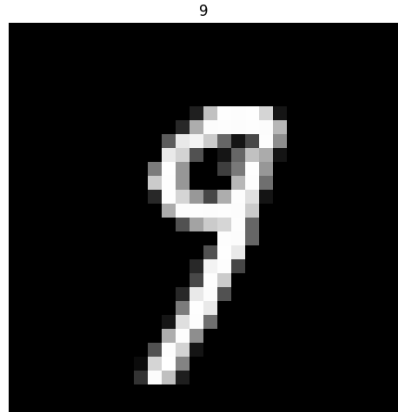
$\odot$



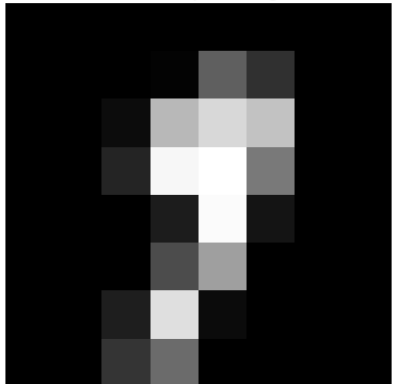
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

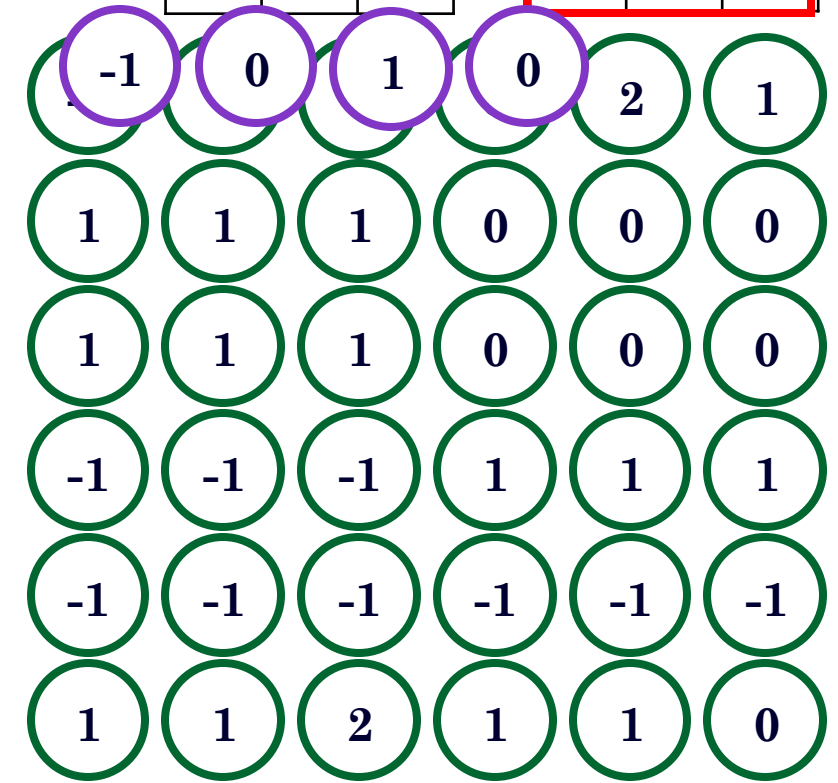
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

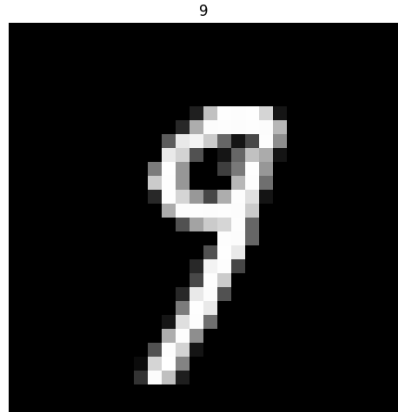
$\odot$



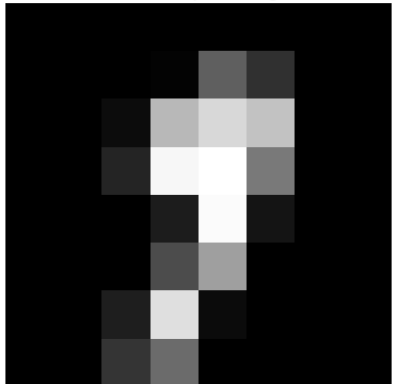
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

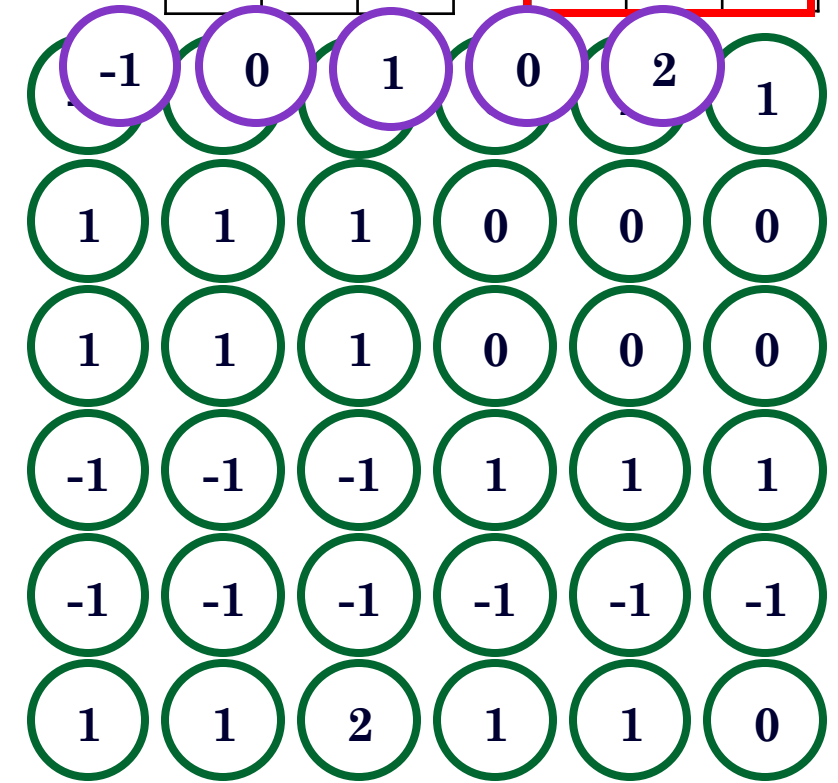
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

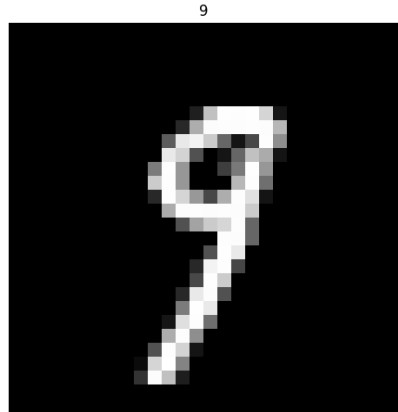
$\odot$



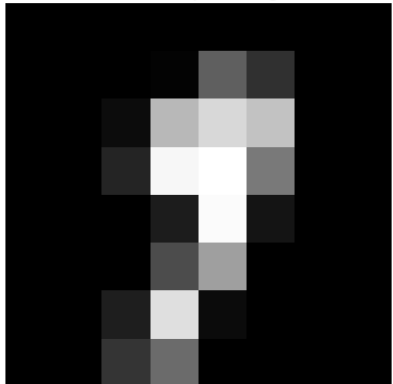
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

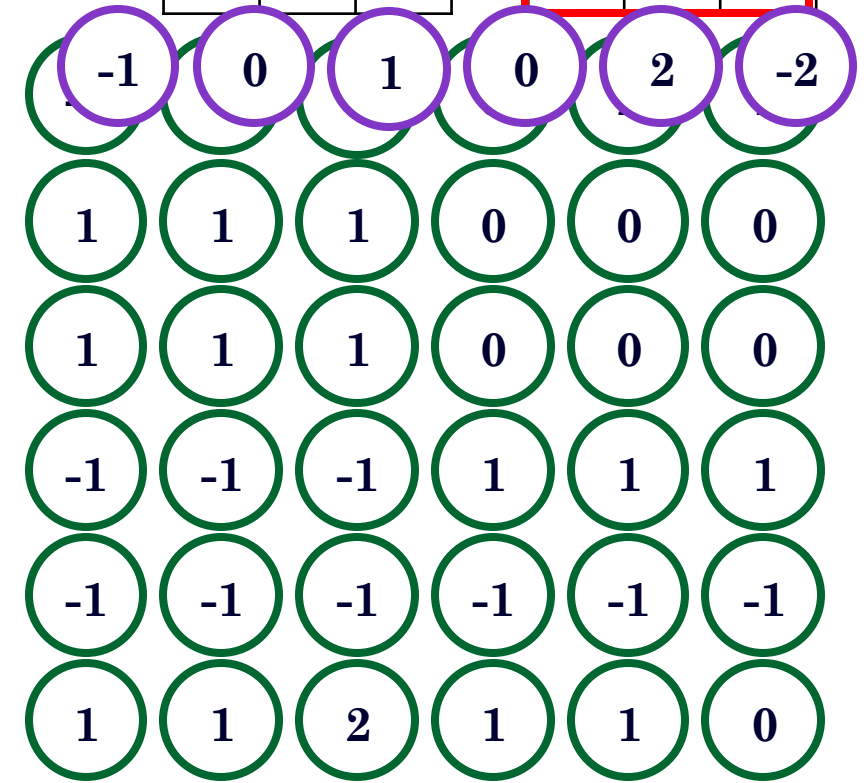
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

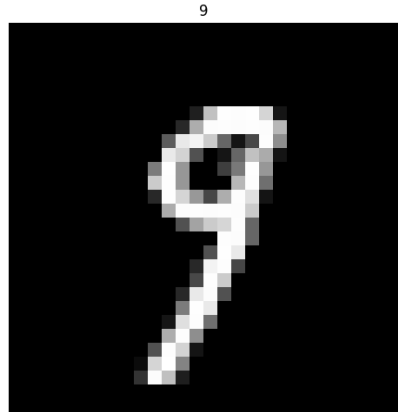
$\odot$



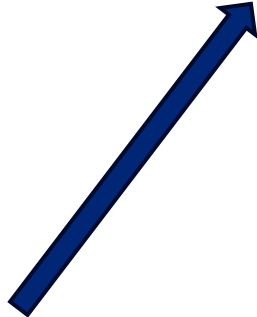
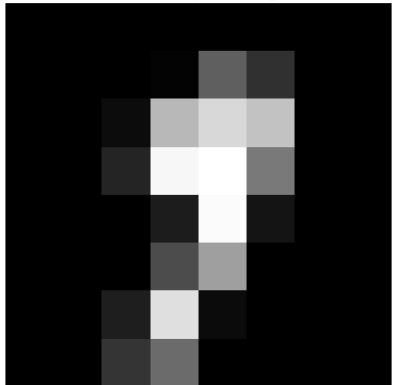
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

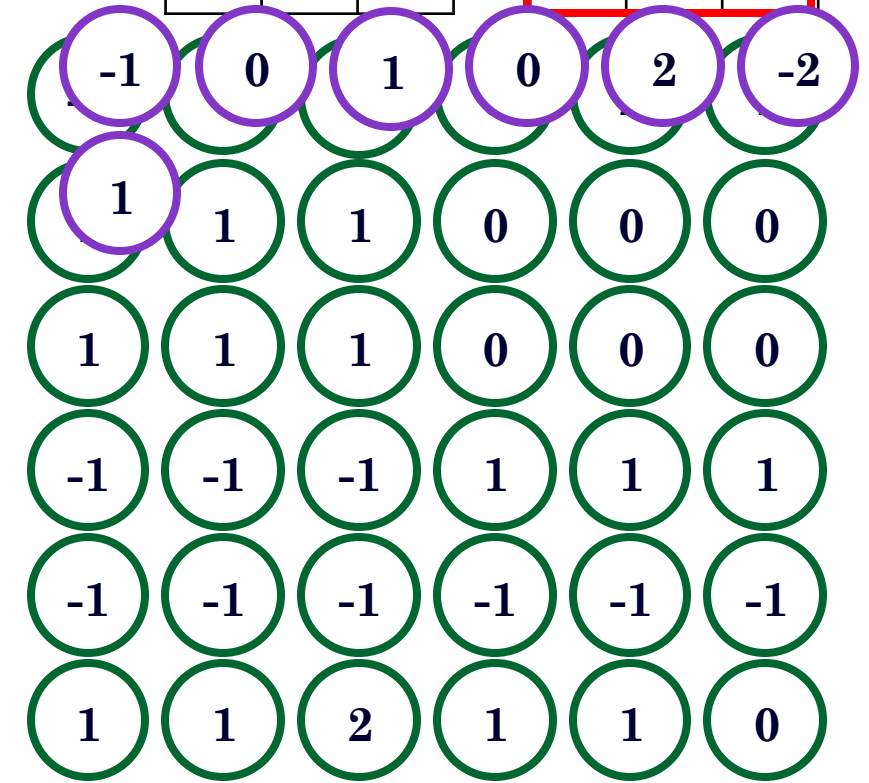
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

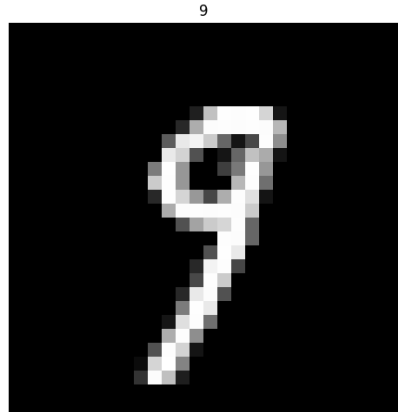
$\odot$



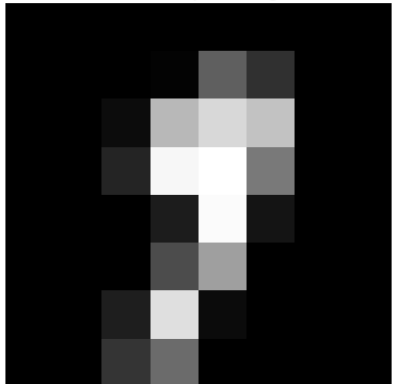
How to Implement Scanning of Features with Weight-Sharing?



1030434



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

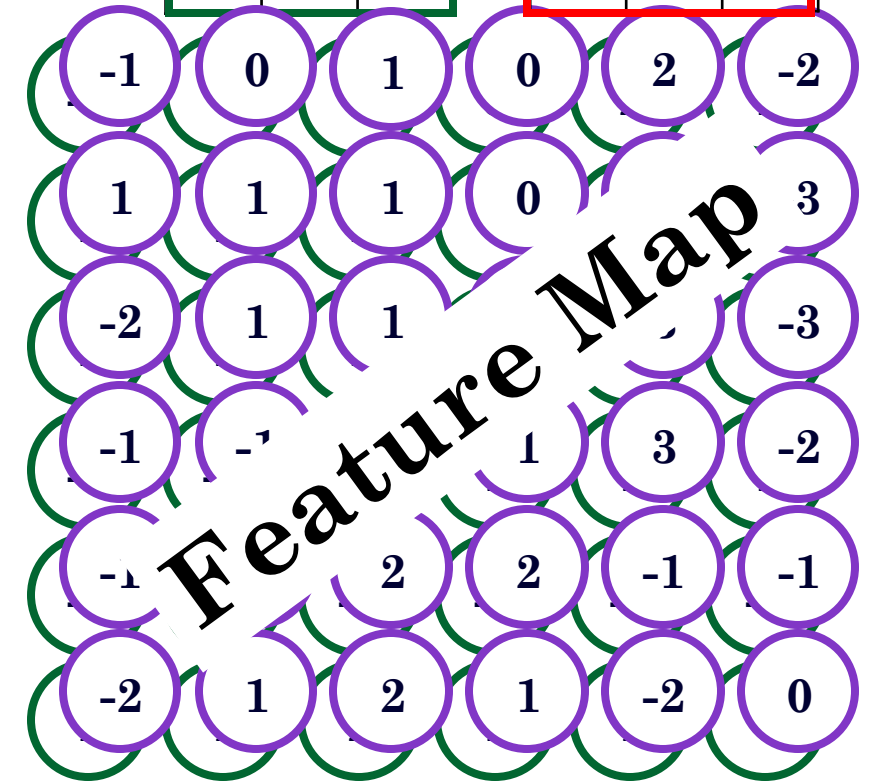
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$

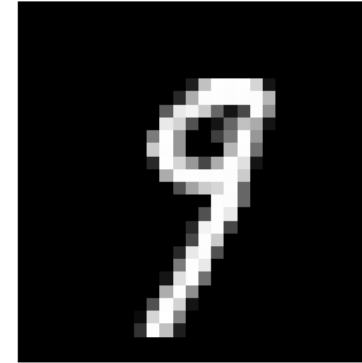


Convolution Example

Kernel: horizontal line



Input Image



Convolved Output



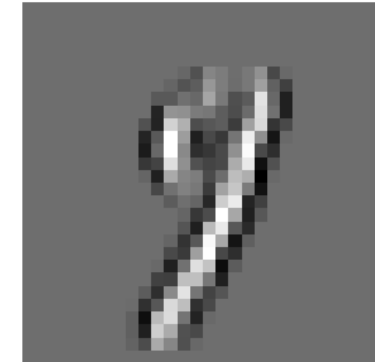
Kernel: vertical line



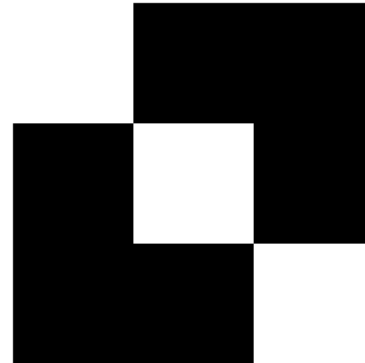
Input Image



Convolved Output



Kernel: diagonal line



Input Image



Convolved Output



1030434



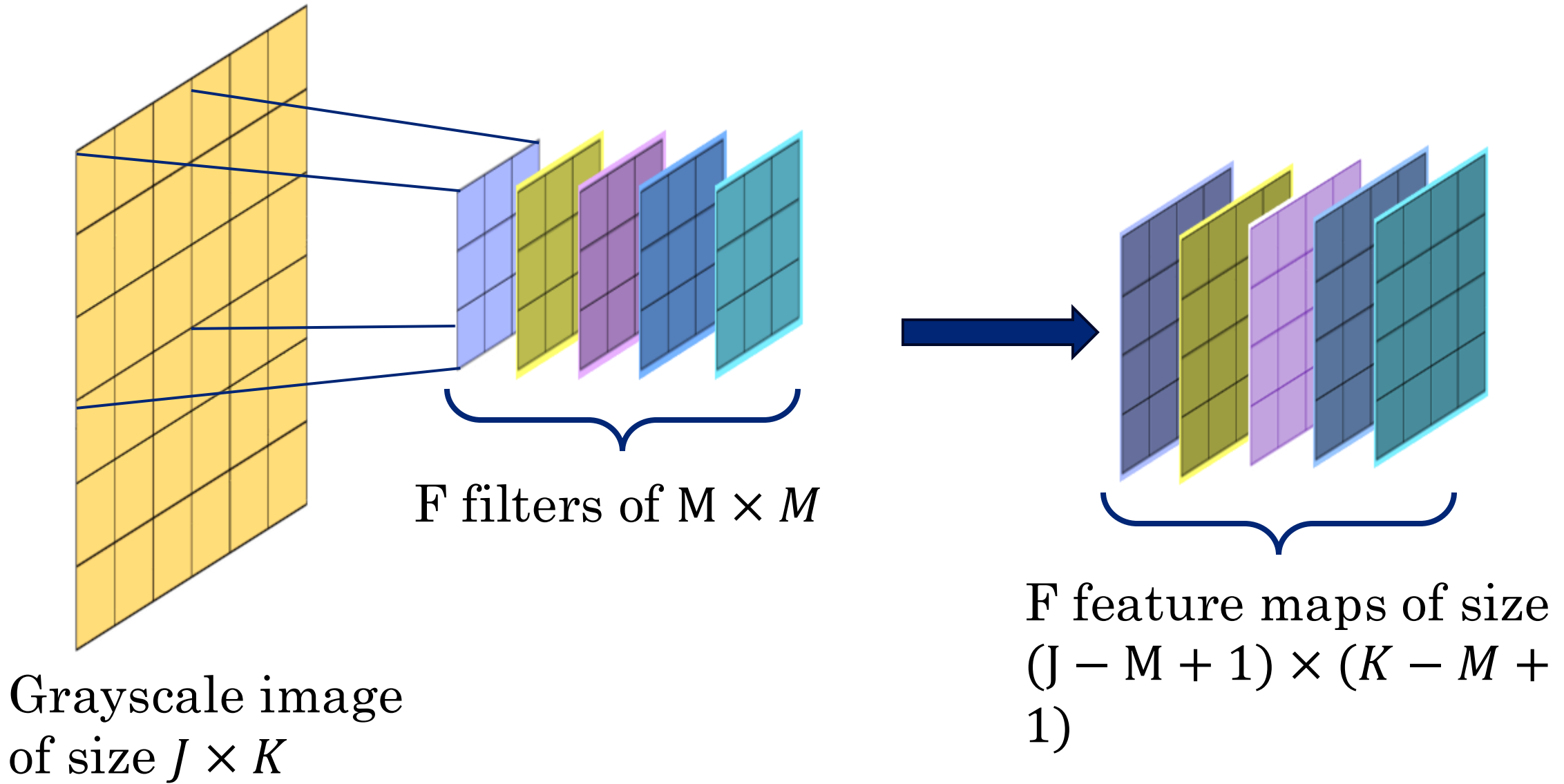
If we convolve a kernel of size 5×5 with an image of size 8×8 with the stride of 1, what are the output dimensions?



If we convolve 4 kernels of size 5×5 with an image of size 8×8 with the stride of 1, what are the output dimensions?

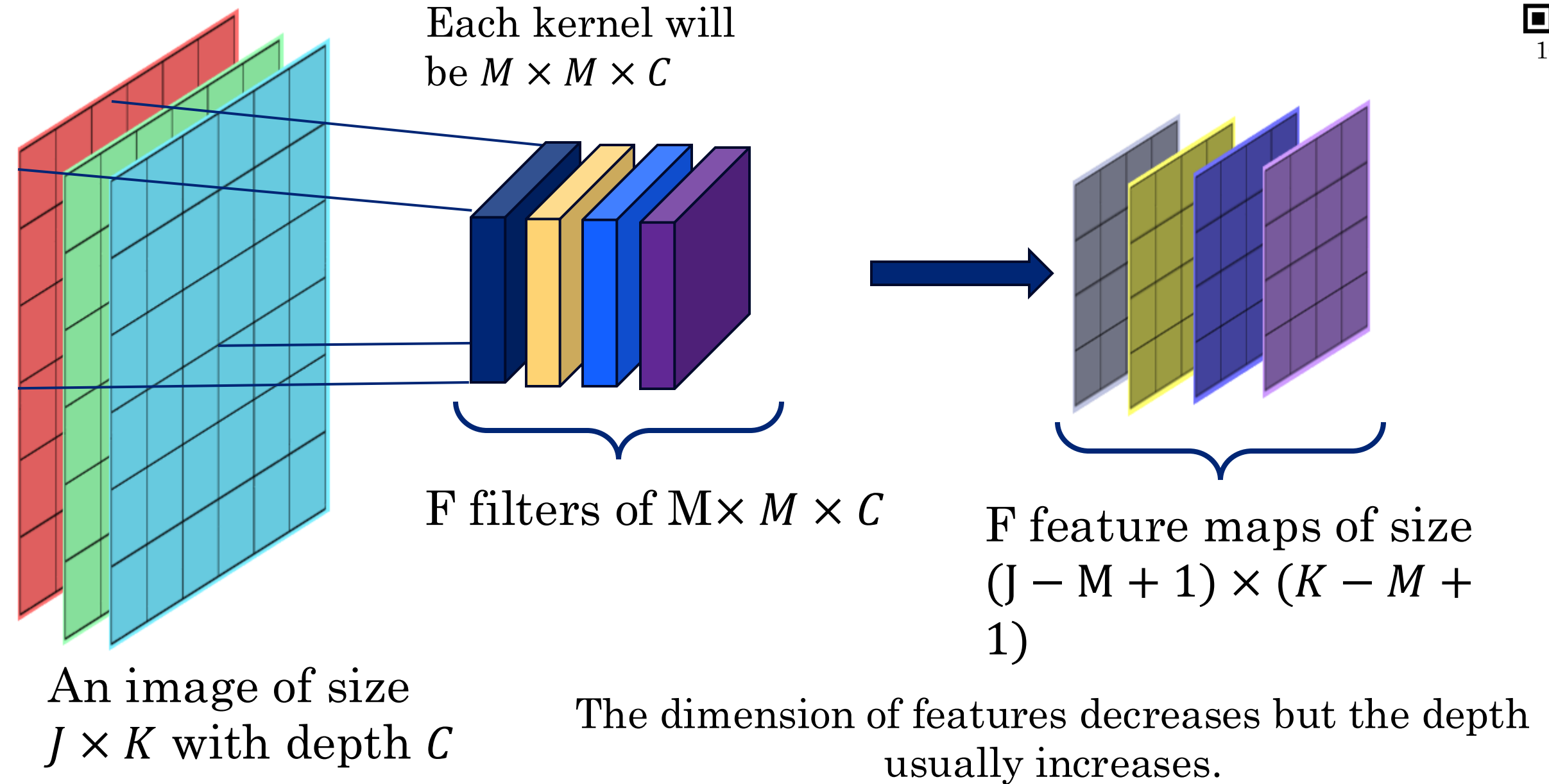


1030434





1030434



What If We Want to Keep the Same Image Dimensions?

- We can use padding.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line
detector

$\odot$

0



1030434

What If We Want to Keep the Same Image Dimensions?



1030434

- We can use padding.

If we pad with P rows and columns on each side, the output dimensions of a

feature map will be:

$$(J + 2P - M + 1) \times (K + 2P - M + 1)$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

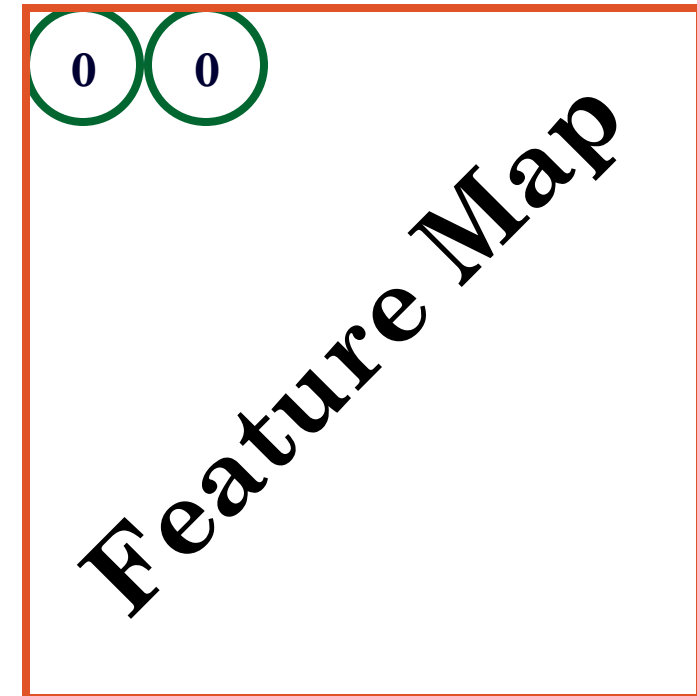
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

Horizontal line detector



Avoiding Redundancy



1030434

- We do not need to move one pixel at a time.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line
detector

$\odot$

0

Stride = 2

Avoiding Redundancy



- We do not need to move one pixel at a time.

If we pad with P rows and columns on each side, and move with the stride of S, the output dimensions of a feature map will be:

$$\left(\frac{J + 2P - M}{S} - 1 \right) \times \left(\frac{J + 2P - M}{S} - 1 \right)$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

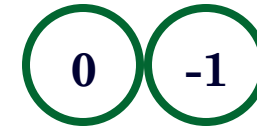
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector

⊙

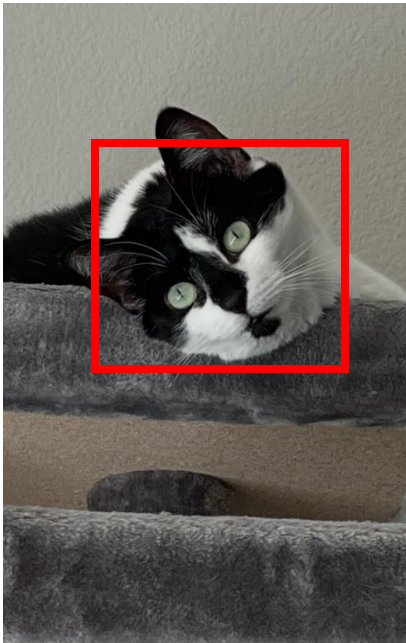


Stride = 2

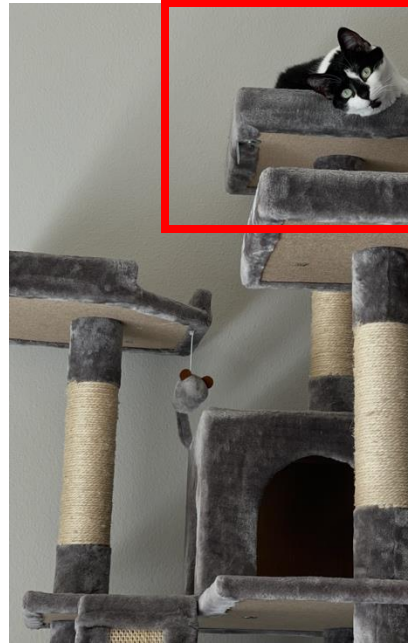
Characteristics of Image Analysis



Patterns are usually smaller than high-resolution images.



Patterns can appear anywhere in the image.

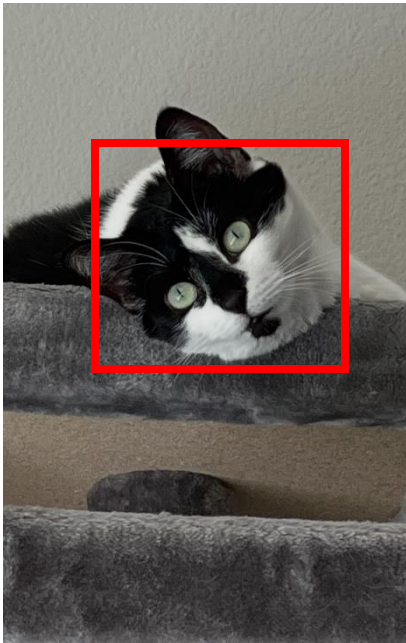


Addressed with
convolution

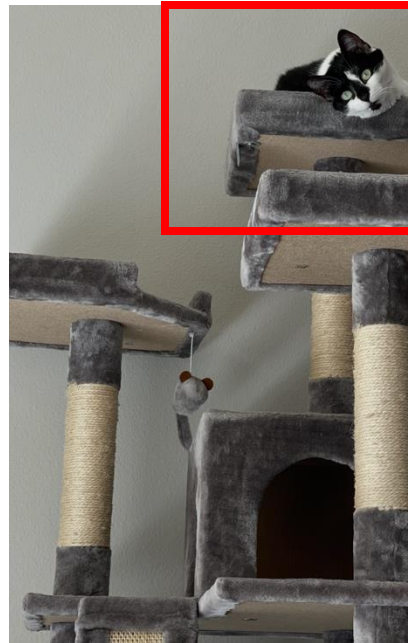
Characteristics of Image Analysis



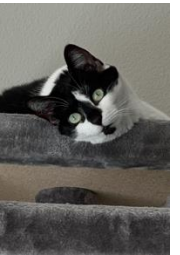
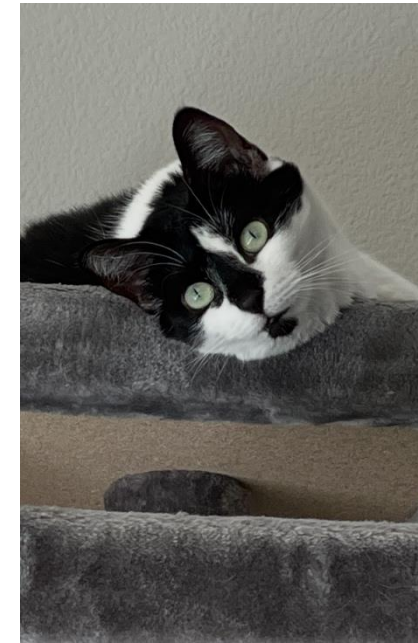
Patterns are usually smaller than high-resolution images.



Patterns can appear anywhere in the image.



Patterns might appear at different scales.



How Can We Detect Features at Different Scale?



1030434

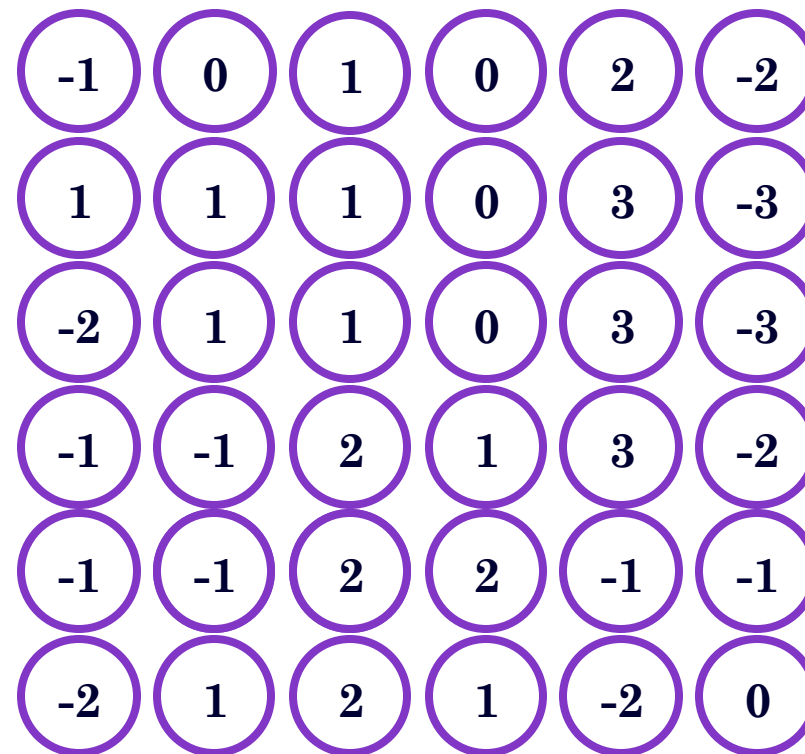
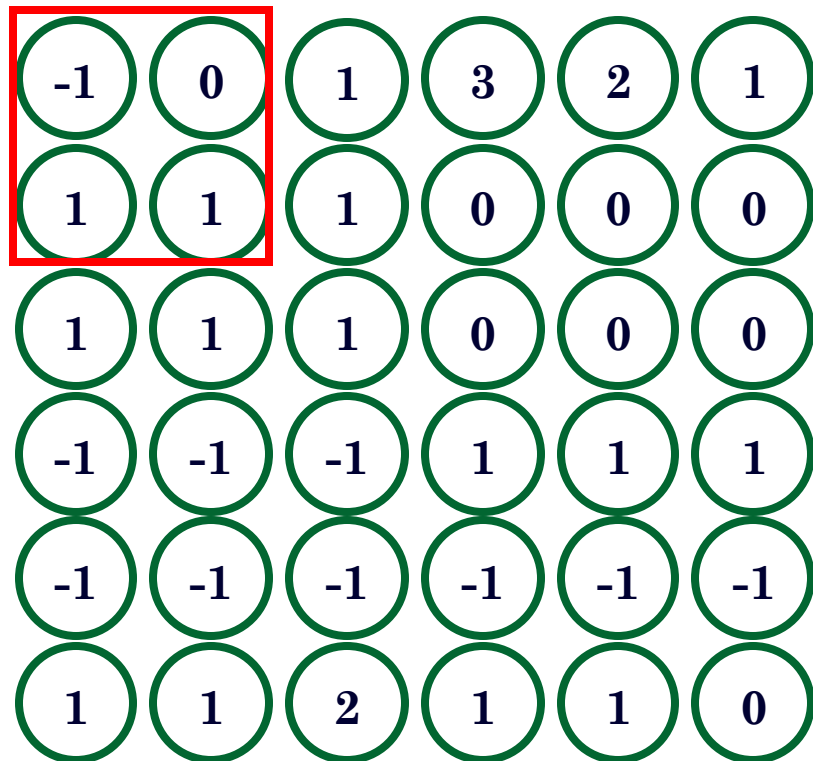
2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

This is called pooling.

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



1030434

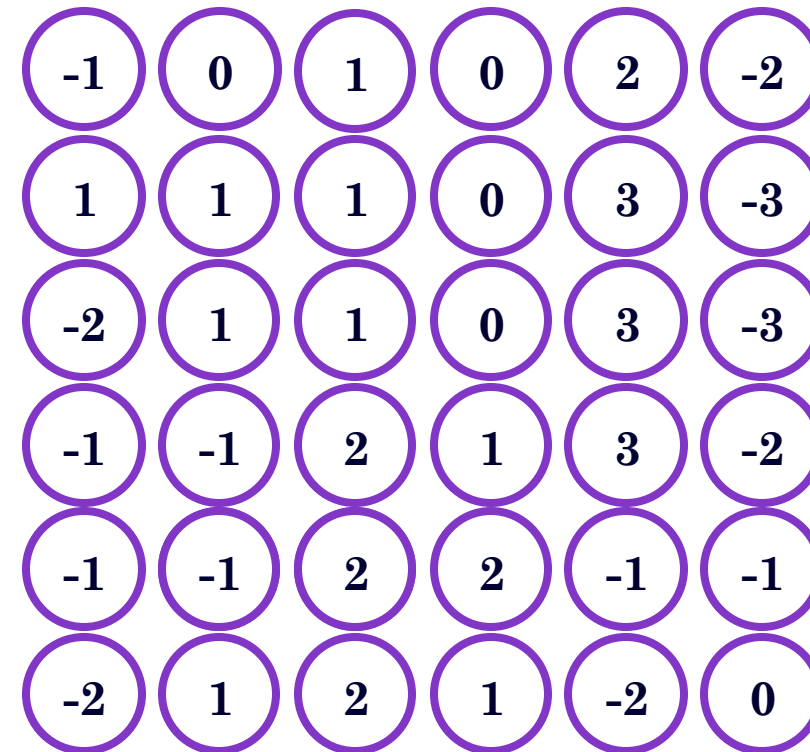
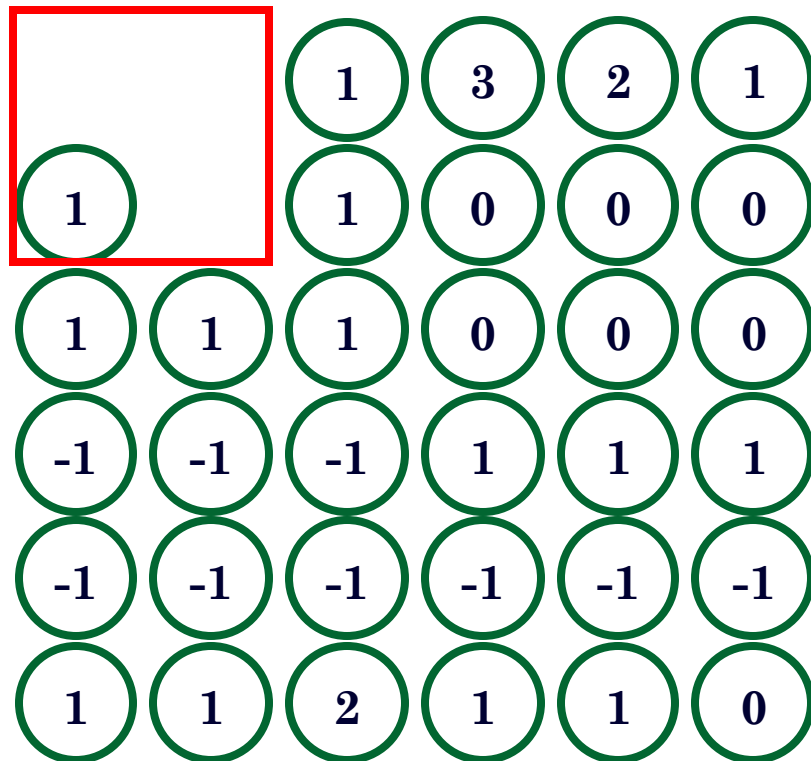
2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

This is called pooling.

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



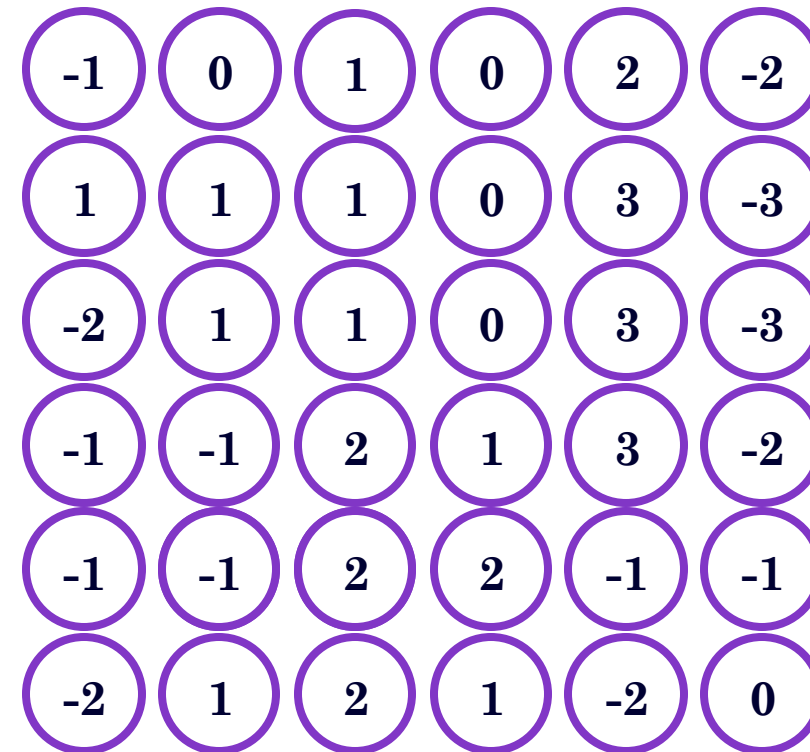
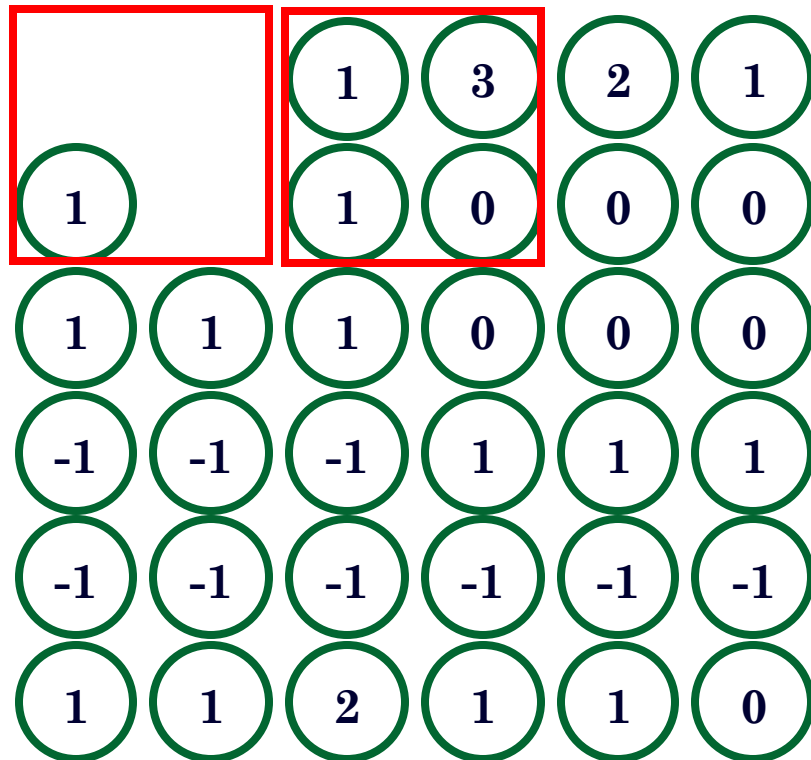
1030434

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



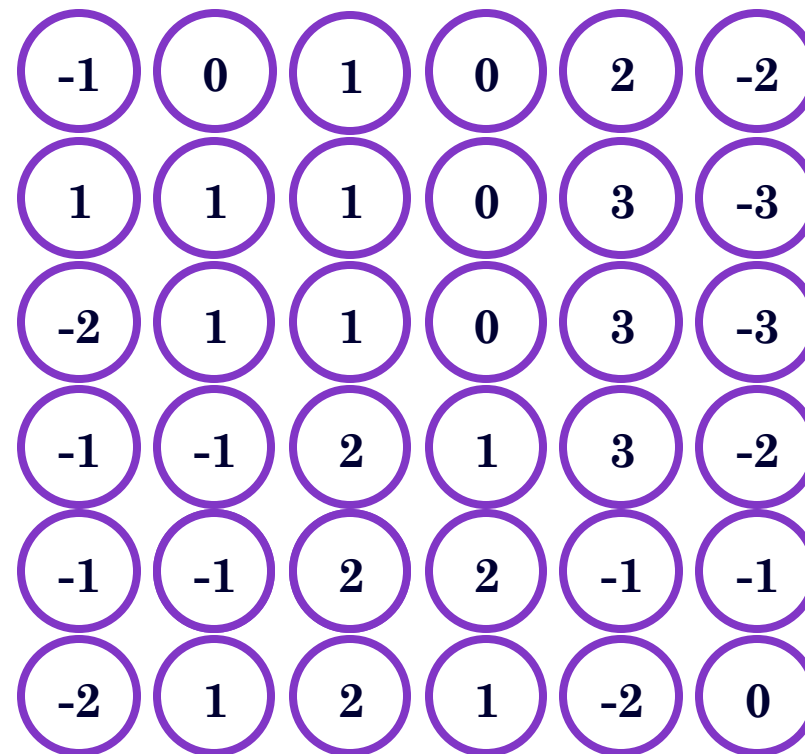
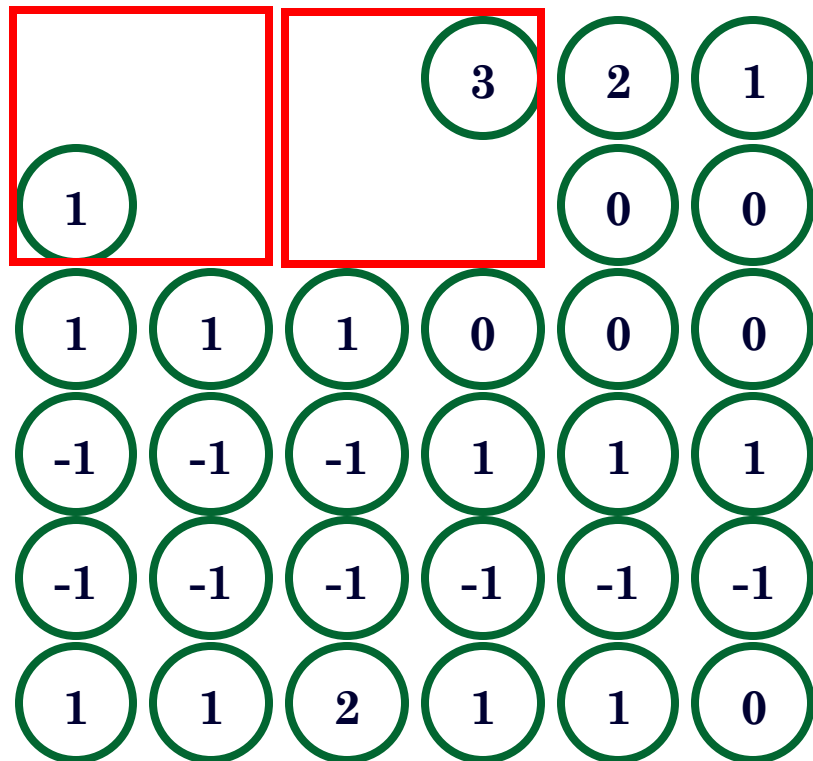
1030434

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



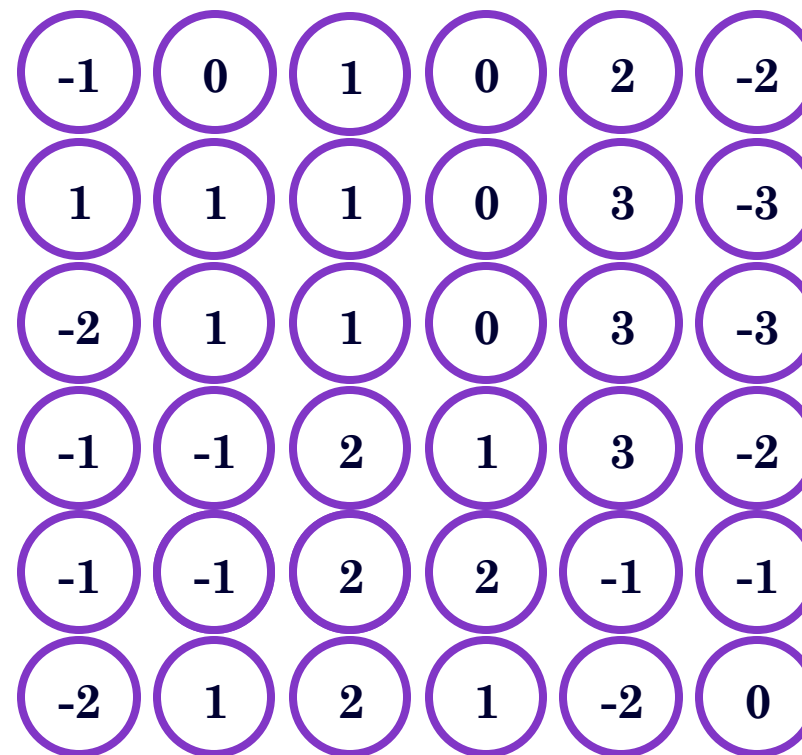
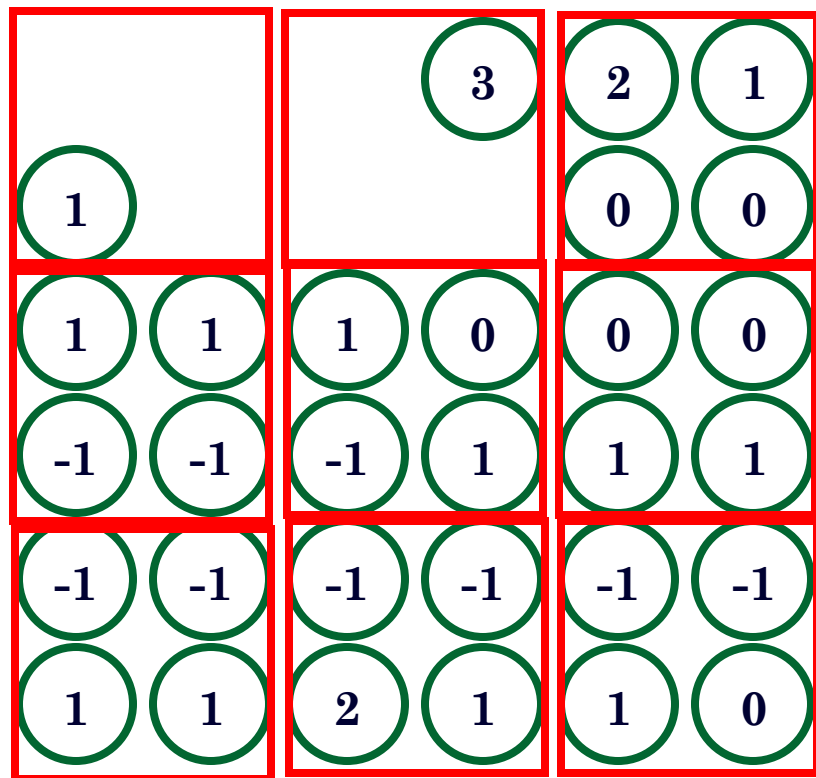
1030434

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



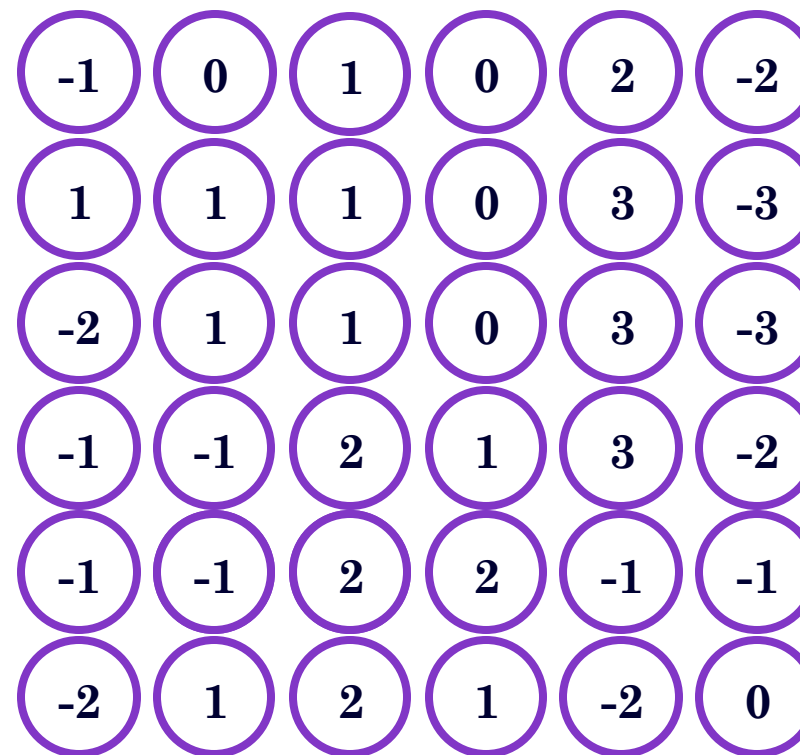
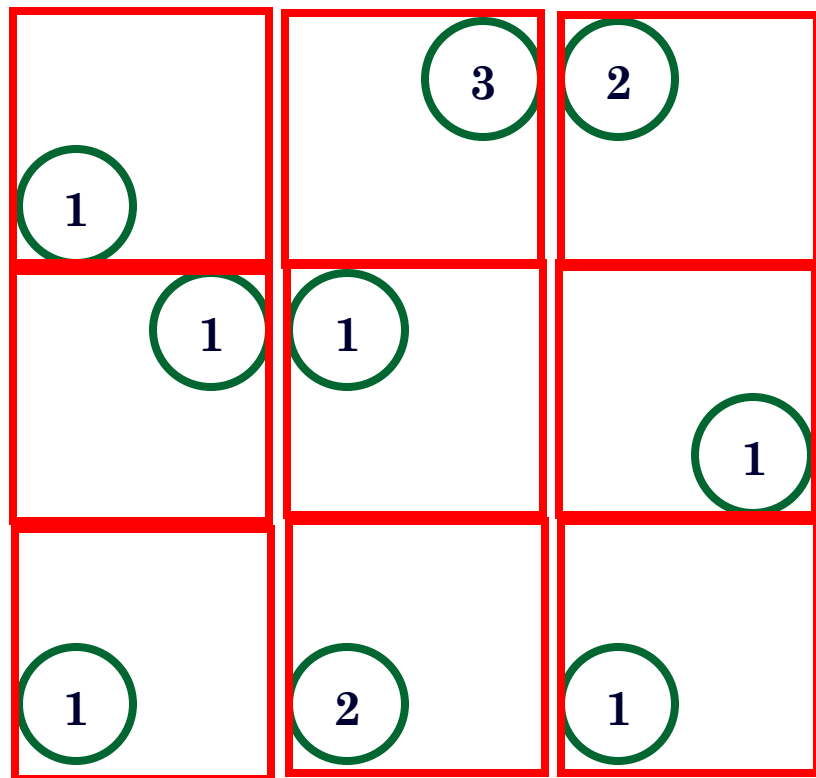
1030434

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



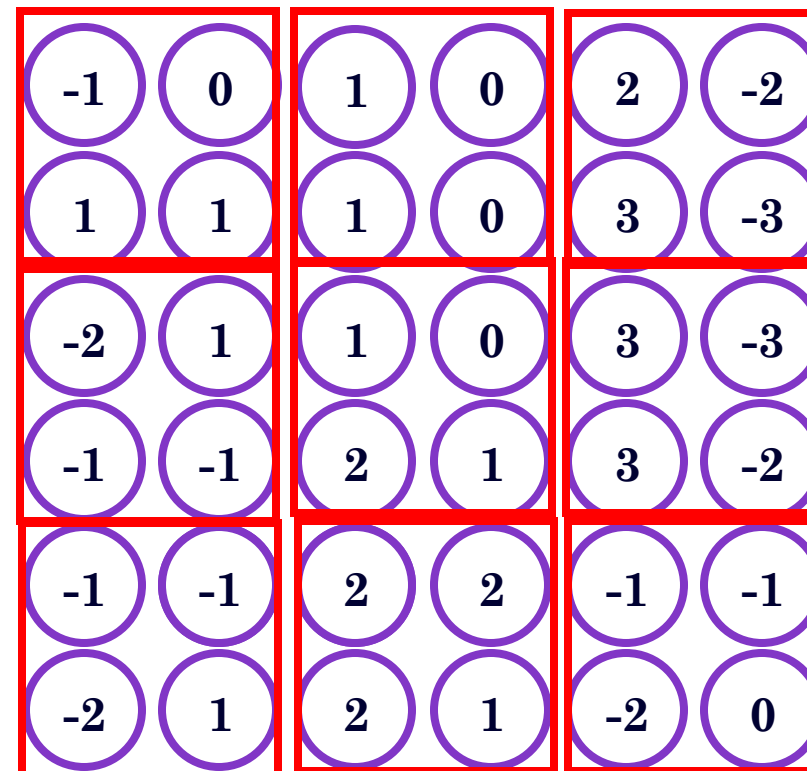
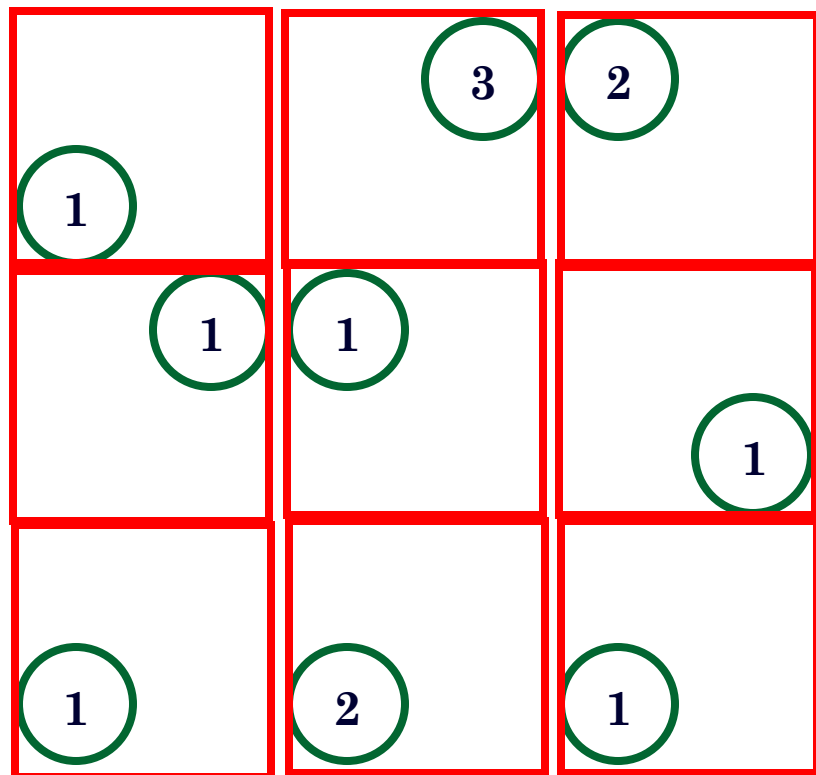
1030434

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



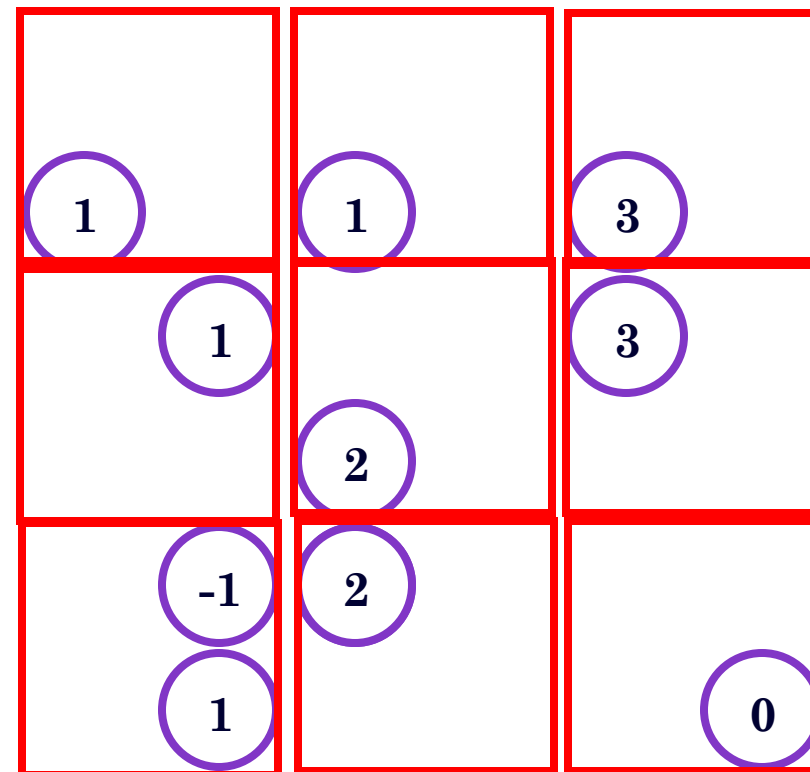
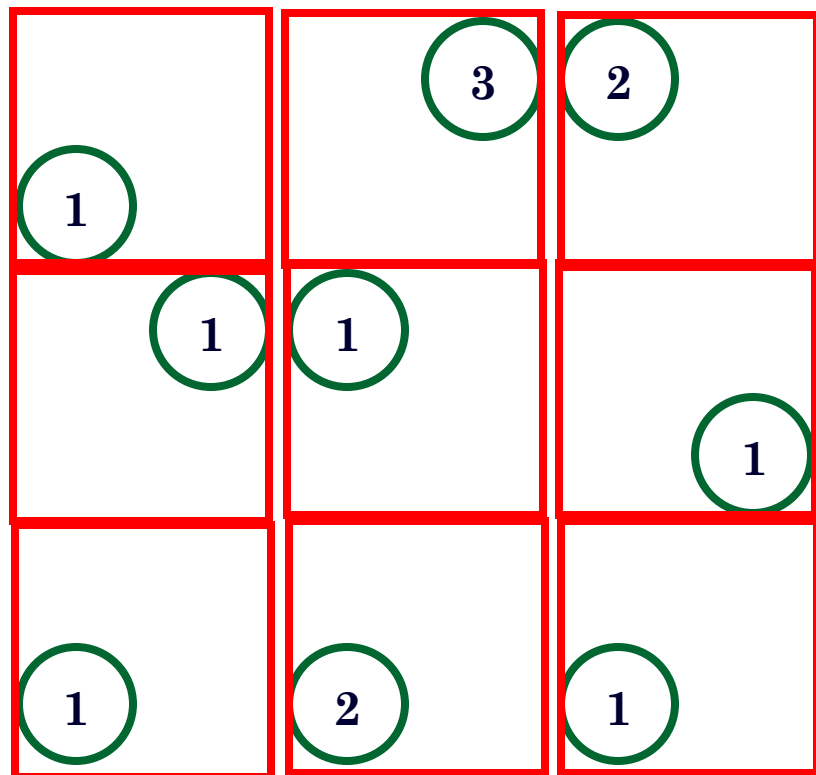
1030434

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



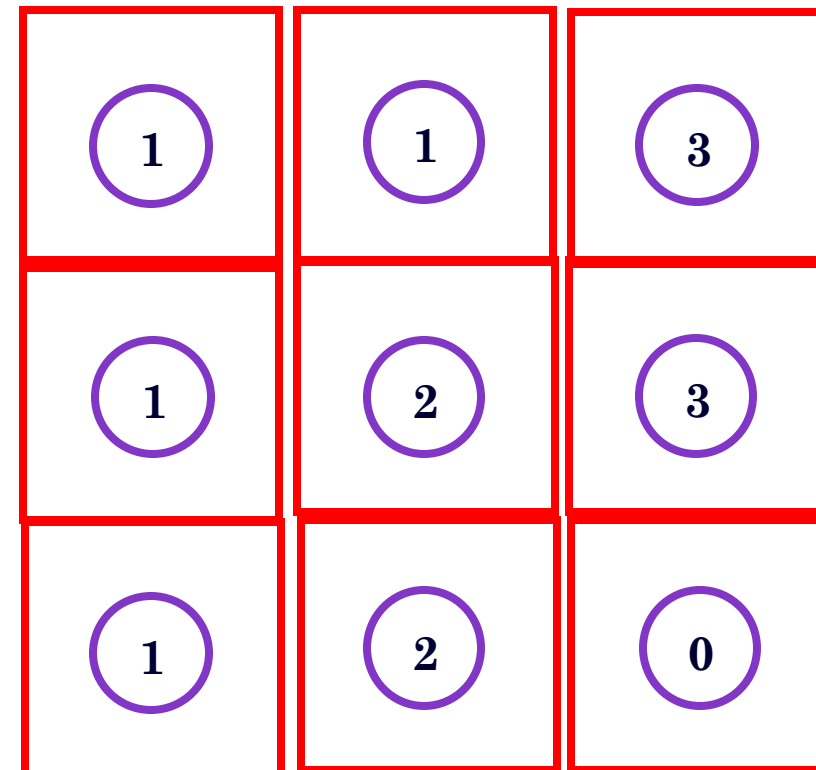
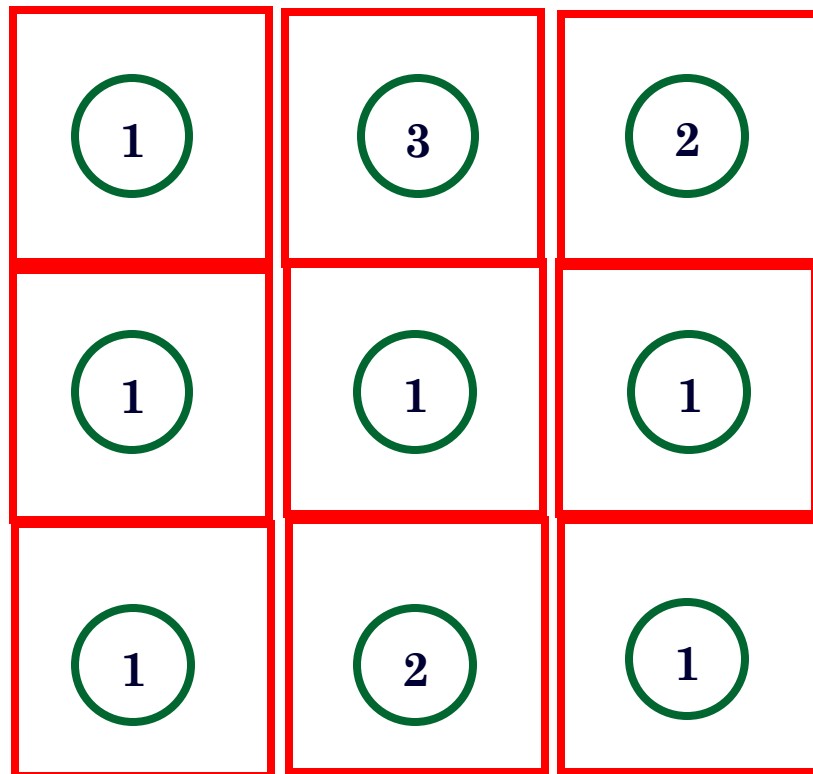
1030434

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



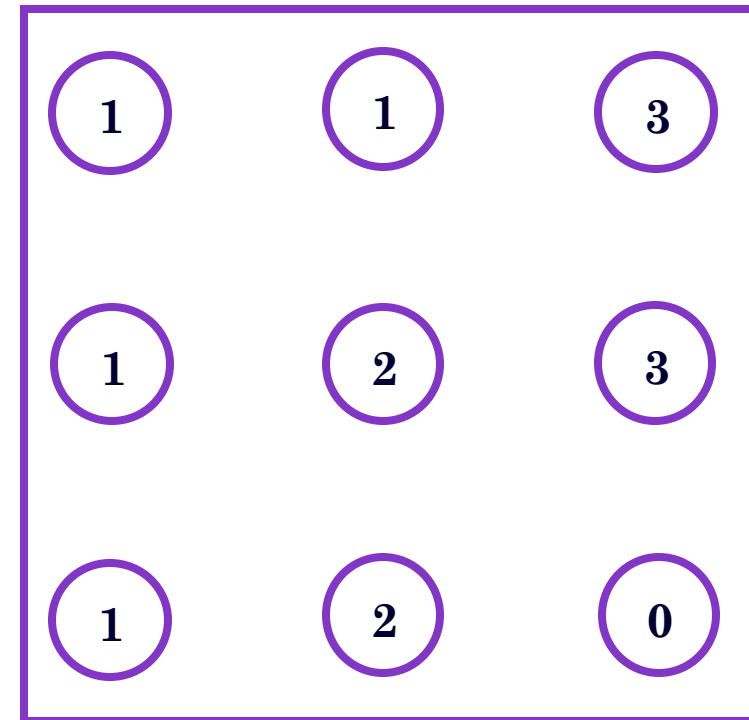
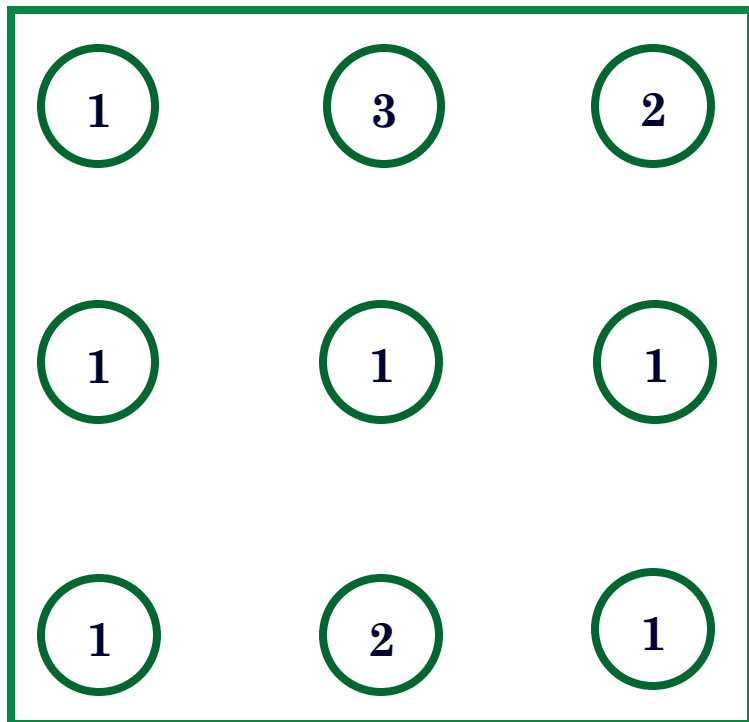
1030434

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

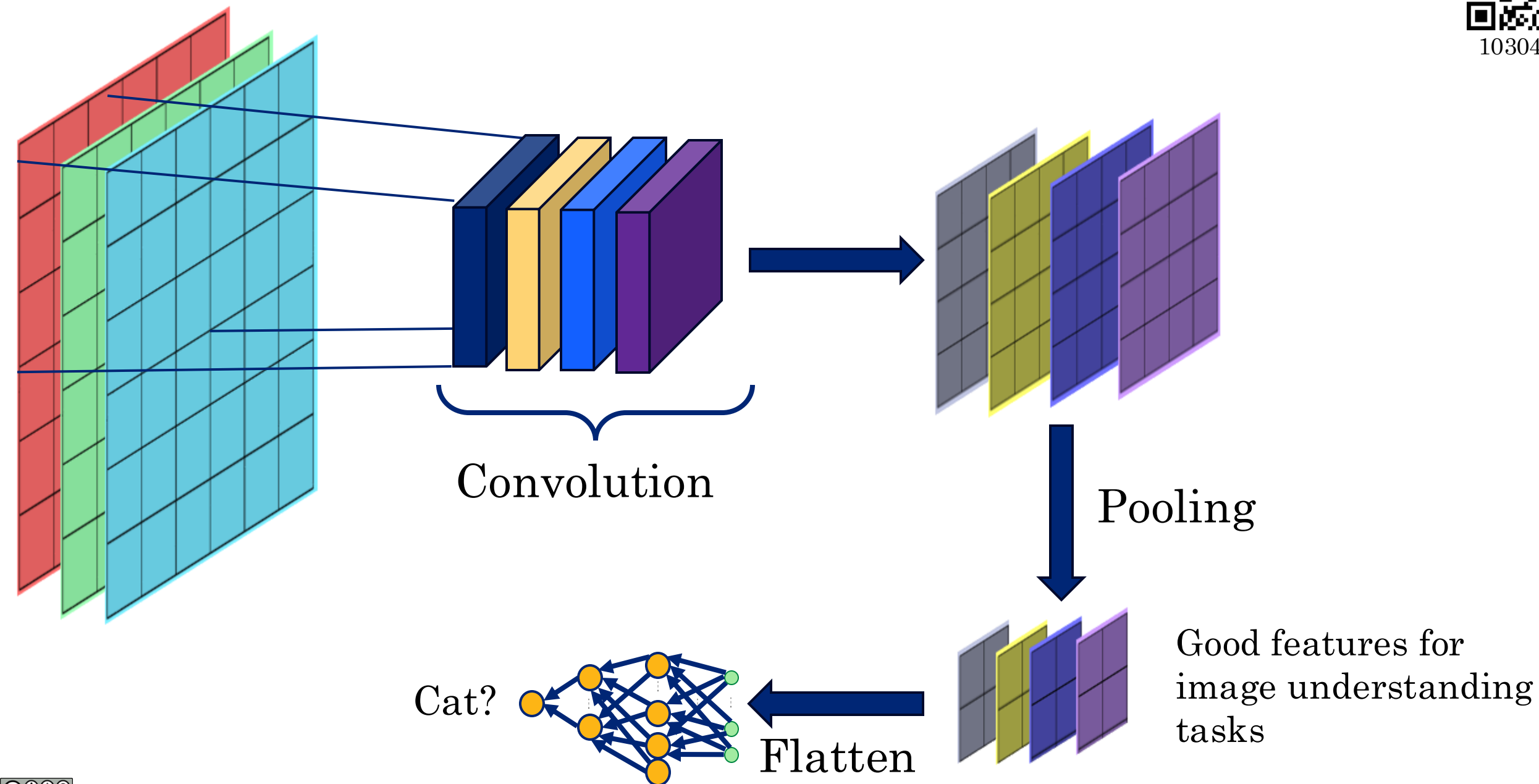
Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1





1030434





1030434

Next Lecture: Building CNN

Lecture 17

Neural Network Regularization & Convolutional Neural Network

Reading: *Chapter 8* in Bishop Deep Learning Textbook and *Chapter 10*